

Agentic-AI Framework for Integrated Design, Implementation, Testing, and Operation of Digital Twin Networks

RAMI KHALDI¹, ARMIN LEHMANN¹, BOGDAN GHITA² (Senior Member, IEEE),
AND ULRICH TRICK¹

¹Research Group for Telecommunication Networks, Frankfurt University of Applied Sciences, 60318 Frankfurt am Main, Germany

²School of Engineering, Computing and Mathematics, University of Plymouth, PL4 8AA Plymouth, U.K.

CORRESPONDING AUTHOR: R. KHALDI (khalidi@e-technik.org)

This article has supplementary downloadable material available at <https://doi.org/10.1109/OJCOMS.2026.3686199>, provided by the authors.

ABSTRACT Digital Twin Networks (DTNs) are emerging as a key enabler for the robust optimization and lifecycle management of next-generation networks. However, current DTN approaches still rely heavily on manual design, implementation, testing, and validation of the DTN system, which are often tailored for specific application domains and lack generalization. This time-consuming creation and adaptation is incompatible with the highly dynamic nature of sixth-generation (6G) networks. Consequently, the DTN must be capable of adapting to the dynamic network environment, necessitating automated DTN generation and operation. Crucially, no unified framework currently supports an agentic workflow covering the entire lifecycle, from use-case inception and requirement specification to operation. This paper addresses this gap by presenting an adaptable, agentic framework to jointly design, implement, test, and operate DTNs in a closed loop. The proposed architecture is designed to be extensible, such that it can be reconfigured for other network domains (e.g., radio access, mobility) via its adaptable configuration mechanisms (e.g., agent prompt engineering, comprehensive data collection and control interfaces). However, both the current implementation and experimental evaluation are scoped to core network management and topology optimization. Subsequently, the framework autonomously orchestrates continuous perception, reflection, planning, code generation, testing, validation, and control. We evaluate the framework's capability through two distinct 6G use cases: Multi-Access Edge Computing (MEC) service migration and network slicing. Extensive evaluations with state-of-the-art Large Language Models (LLMs) demonstrate the framework's ability to reliably translate high-level intents into valid actuations that fulfill the specified Key Performance Indicators (KPIs), achieving high success rates while revealing distinct model-specific engineering strategies.

INDEX TERMS Agentic-AI, closed-loop control, digital twin network (DTN), generative artificial intelligence (GenAI), intent-based networking (IBN), network management, requirements engineering, sixth-generation (6G) network.

I. INTRODUCTION

THE concept of Digital Twin, initially established in Product Lifecycle Management (PLM) for real-time monitoring and optimization [1], is now crucial to the evolution of telecommunications. In the networking domain, Digital Twin Networks (DTNs) represent a paradigm shift from static simulation to dynamic, bidirectional synchronization between physical infrastructure and its virtual representation. By maintaining a high-fidelity, real-time replica of the physical network, DTNs enable

closed-loop automation for continuous network optimization, while offering granular insights into the network state [2], [3], [4].

A reference DTN architecture is illustrated in Fig. 1. The DTN continuously pulls telemetry (data collection) from the physical network layer at the bottom and exerts control over it via southbound interfaces to optimize it. Conversely, the application layer at the top communicates intents through northbound interfaces, through which the DTN also exposes its capabilities. The middle layer constitutes the core of the

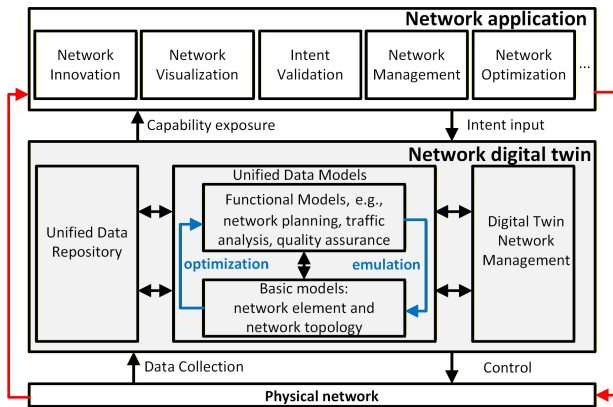


FIGURE 1. Reference DTN architecture [2], [5].

DTN architecture and comprises three principal subsystems. The Unified Data Repository supports data acquisition, storage, and service exposure. The Digital Twin Network Management subsystem handles model, topology, and security management. Finally, the Unified Data Models component includes both basic models (network-element and topology models) and functional models such as network planning, traffic analysis, and quality assurance [2], [5]. The architecture supports a double closed-loop mechanism. The inner closed loop (shown in blue) operates within the digital twin layer, where basic and functional models interact to support optimization and emulation. The outer closed loop (shown in red) spans all three layers, coupling telemetry from the physical network with intent input from applications to enable continuous control, feedback, and real-time network optimization.

The DTN should not be viewed as a static framework that is merely designed, implemented, and operated to fulfill fixed network requirements. Instead, the highly dynamic and heterogeneous nature of future 6G networks requires a DTN that evolves with the physical network and its operational objectives. For example, existing models may require adaptation, or entirely new models may need to be generated when the current DTN instance fails to satisfy specific requirements. Therefore, the DTN must be capable of automated adaptation to ensure continuous alignment with both the physical infrastructure and the overlying service demands. Despite this critical requirement, most research often treats DTN construction as a static, pre-operational phase, a limitation that impedes the ability to address the fluid and complex requirements of 6G systems. In addition, there is a lack of frameworks capable of empowering Artificial Intelligence (AI) agents to actively *design, implement, test, and operate* the DTN for diverse network use cases in an automated manner.

This paper is organized as follows. Section II reviews the related work. Section III defines the problem statement and design objectives. Section IV presents the proposed agentic DTN framework architecture. Section V details the system implementation. Section VI evaluates the framework through two use cases. Section VII discusses the benefits,

limitations, operational considerations, and lessons learned. Finally, Section VIII concludes the paper and outlines future work.

II. RELATED WORK

The integration of Large Language Models (LLMs) into network management is evolving from intent translation to exploring multi-agent collaboration for specific networking tasks. This section reviews the state of the art, categorizing contributions into intent-based management, multi-agent orchestration and collaboration, generative AI for DTN operations, domain-specific applications, and surveys and theoretical frameworks. We then position this work by addressing identified gaps.

A. INTENT-BASED NETWORK MANAGEMENT

Early works in the network domain focused on using LLMs to translate high-level human intents into low-level configurations. Jeong et al. [6] proposed S-Witch, an assistant for switch configuration based on prompt engineering. While they validate commands in a digital twin, the authors report difficulties with topology-dependent tasks like IP address allocation and complex configuration details, highlighting the limitations of pure intent translation without a holistic design framework. Similarly, Dzevaroska and Leon-Garcia [7] used a generic LLM with in-context learning capabilities, combined with feedback controllers, to provide continuous Key Performance Indicator (KPI) assurance and corrective actions. However, they explicitly discuss limitations regarding generalizability, scalability, and the lack of standardized datasets. Furthermore, they highlight that ensuring the accuracy, robustness, and trustworthiness of the generated management actions remains a critical open challenge.

B. MULTI-AGENT ORCHESTRATION AND COLLABORATION

To handle the complexity of modern networks, multi-agent architectures have emerged. Xu et al. [8] proposed a split learning system for deploying AI agents in 6G networks, where perception, grounding, and alignment modules were distributed across mobile devices and edge servers, but this work focuses on the integration of specific AI agents to tackle narrow use cases. Chen et al. [9] presented a 6G agent prototype that utilizes LLMs to configure network resources, such as spectrum allocation and Service Level Agreement (SLA) assurance. Although they identify agent misbehavior as a critical risk requiring Digital Twin monitoring, their current prototype lacks a closed-loop sandbox validation mechanism to verify actions before actuation. Xiao et al. [10] proposed task-driven LLM agents tightly integrated with digital twins for 6G system optimization. While promising, this work relies on specialized fine-tuning and is limited to physical layer optimization. Despite these advancements, across these multi-agent frameworks, agents primarily orchestrate or optimize pre-existing network functions. They do not autonomously design the DTN internal logic, select or instantiate models, or construct the software stack.

In the domain of cross-domain orchestration, Xu et al. [11] proposed a framework using multi-agent LLMs to facilitate interoperability across IP networks, optical, and robotic domains, but their approach relies on fine-tuned models, which are resource-intensive and require domain expertise, and focuses on the orchestration of pre-defined tasks within existing domains. Zaid et al. [12] introduced a multi-agent LLM-assisted network controller utilizing general-purpose models without fine-tuning, yet their framework is specifically tailored for data privacy compliance in network management, rather than being a generic solution. Furthermore, it does not address the autonomous design and implementation of the underlying DTN architecture. Cui et al. [13] introduced LLMind, a framework for distributed Internet of Things (IoT) device collaboration, which is tailored for task execution rather than generic network system design.

C. GENERATIVE AI FOR DTN OPERATIONS

DTNs are increasingly augmented with LLMs to enhance interactivity and decision-making. Tao et al. [14] explored the potential of generative AI as a catalyst for 6G DTNs, identifying key application areas, yet their work remains a high-level architectural discussion and does not offer a specific, implemented framework for the autonomous design, construction, and operation of the DTN system. Shu et al. [15] presented a vision for digital-twin-enabled network autonomy, but their work remains largely conceptual. Corici et al. [16] introduced the concept of a “Generative Twin” for 6G, utilizing generative AI to create system configurations and test scenarios. Although they present an implementation of this generative approach, it is primarily tailored to optimizing specific network configurations and test cases. Crucially, it does not address the fully autonomous *design and implementation of the DTN software stack*, where agents actively select, adapt, and interconnect models to build the DTN from scratch. Hong et al. [17] proposed LLM-Twin for beyond 5G, focusing on a “mini-giant” model collaboration scheme to enable semantic-level secure communication and efficient multimodal data processing. Nonetheless, their contribution is primarily limited to optimizing the operational data exchange and security. Similarly, Jiang et al. [18] introduced LINKs, an LLM-driven framework that leverages multi-agent collaboration to autonomously optimize data retrieval and radio resource management in 6G-enabled DTNs, specifically targeting smart city scenarios. Although their approach achieves autonomous problem analysis and resource allocation, it is fundamentally an operational optimization framework. It assumes a pre-existing DTN structure and does not address the autonomous *design, implementation, and instantiation* of the DTN stack. Yang et al. [19] elaborated on large generative models (LGM) for 6G networks, proposing an LGM-enabled digital twin framework specifically designed for intelligent, automated network control and customized services. Nevertheless, similar to previous approaches, it focuses on utilizing the DTN for operational tasks rather than addressing the autonomous *design and*

implementation of the DTN system. While this framework advances the integration of LLMs and DTNs, it typically treats the DTN as a static tool for specific control tasks.

One other strand of recent research in relation to DTN focused on optimization frameworks. Zeeshan et al. [20] utilized an LLM-based Retrieval-Augmented Generation (RAG) framework for resource optimization, identifying the high computational overhead and latency of LLMs as critical challenges for real-time wireless applications, while Haider et al. [21] introduced LLM-DTNet, utilizing fine-tuned task-specific AI models and LLMs for hierarchical resource allocation. Long et al. [22] proposed an LLM-based architecture for 6G network operations and optimization. However, these works primarily target operational tasks. Regarding the early design phase, Blasek et al. [23] investigated LLMs for digital twin requirements engineering, but their study was limited to manual interactions with ChatGPT-4, noting issues with response consistency and a lack of deep domain knowledge, and did not close the loop to implementation and operation.

D. DOMAIN-SPECIFIC APPLICATIONS

Specific applications include autonomous optical networks, where Song et al. [24] and Zhang et al. [25] utilized LLMs as cognitive agents for operational management and monitoring. Despite their effectiveness, these solutions lack transferability to other network domains. Zhou et al. [26] demonstrated an LLM-driven agent integrated into a Software-Defined Networking (SDN) controller, using a digital twin of optical networks for performance prediction and optimization. Nevertheless, their use case is narrowed to the SDN controller. Additionally, they note that enabling LLMs to process the diverse data types requires further investigation. Collectively, this approach lacks a holistic framework that manages the entire lifecycle, particularly the automated design and implementation of the management software. Similarly, Yu et al. [27] introduced ChannelGPT, utilizing a fine-tuned large model, carrying the associated challenges, to focus exclusively on wireless channel modeling and generation, rather than the holistic design and implementation of the complete DTN system. Liu et al. [28] explored the synergy between generative AI and data center networking, offering a domain-specific perspective.

Although effective in their respective verticals, these approaches do not generalize across domains and do not address the end-to-end lifecycle required for autonomous DTN construction and operation.

E. SURVEYS AND THEORETICAL FRAMEWORKS

Sheraz et al. [29] provided a comprehensive survey of AI-enabled digital twin networks for 6G. However, their review primarily focuses on the application of AI for specific network optimization tasks within established DTN architectures. Recent surveys by Li et al. [30] and Qin et al. [31] highlighted the growing intersection of generative AI and DTNs in telecommunications. Zhou et al. [32] provided

a comprehensive survey on LLMs for telecommunications, yet their work missed emphasizing the importance of LLM-driven DTNs. Common to these works is that the applications reviewed are tailored to specific verticals. They lack a generalized approach that leverages a structured model knowledge base to build DTNs for arbitrary scenarios, and they do not cover the full feedback loop from requirements engineering to operation.

F. DISCUSSION AND POSITIONING OF THIS WORK

While these studies demonstrate the synergy between LLMs and DTNs, existing work remains preliminary and too narrow in scope. Fundamentally, these approaches assume a pre-existing, static DTN and focus solely on using it for operational optimization. None provide a mechanism for dynamically constructing the DTN based on a network use case on demand, where adaptable, yet network-specific agents derive requirements, perceive the environment, reflect on the state, design and implement a solution, verify it in virtual space, execute actions, validate the resulting actuation, and correct themselves in a closed loop. Current approaches primarily focus on the *interaction* with an existing DTN or target specific, narrow vertical applications, often relying on tailored LLMs solutions for isolated tasks. Consequently, there is no unified framework that supports an agentic mode covering the entire lifecycle, from use-case inception and requirement specification to operation, in an agentic manner. Moreover, evaluations in the literature are frequently limited to isolated runs without statistical significance, and the explicit connection to broader 6G architectural requirements is often missing. Most critically, there is a distinct absence of frameworks that empower AI agents to actively *perceive, reflect, design, implement, test, and operate* the DTN across diverse network use cases.

A prerequisite for automated DTN generation and operation is a structured understanding of DTN models. In previous work [33], we provided a taxonomy and comparative analysis of DTN models and representation methods, establishing a foundation for selecting appropriate modeling techniques. The present work builds on these insights as well as earlier research on DTN applications and requirements [4] and use-case-driven DTN design [34]. This paper addresses the above identified gaps by presenting an adaptable, agentic framework to jointly design, implement, test, and operate DTNs in a closed loop.

III. PROBLEM STATEMENT AND DESIGN OBJECTIVES

Despite the rapid advancements in LLM and DTN technologies, the implementation of fully autonomous, self-evolving networks remains hindered by several fundamental challenges. Current approaches typically treat the DTN as a static framework that is manually engineered, limiting its ability to adapt to dynamic 6G use cases. Furthermore, the integration of generative AI into critical infrastructure raises significant concerns. This section identifies the key challenges and design objectives the framework aims to meet.

A. PROBLEM STATEMENT

Building on the related work and its limitations, we identify the key challenges in this subsection.

1) STATIC AND MANUAL DTN LIFECYCLE

Existing DTN implementations are predominantly static. They are designed and built by human experts for a specific purpose and lack the capability to evolve autonomously. The process of translating high-level operational intents into deployable network artifacts remains heavily manual, error-prone, and slow. There is currently no unified framework that automates the entire lifecycle, from the initial use-case inception and requirement specification to the iterative design, implementation, testing, and continuous operation.

2) TRUSTWORTHINESS, HALLUCINATIONS, AND NON-DETERMINISM

LLMs introduce the risk of hallucinations [41], where models could generate misconfigurations. Furthermore, LLMs are inherently non-deterministic: identical prompts can yield vastly different outputs across multiple runs or different models. Deploying unverified AI-generated code without a rigorous validation mechanism can lead to service outages or security breaches. Most existing literature focuses on open-loop intent translation.

3) LACK OF GENERALIZATION

Current AI-driven network solutions are often tailored to specific verticals or narrow tasks. They frequently rely on resource-intensive fine-tuning of models for specific scenarios, creating rigid systems that cannot adapt to new domains without significant retraining. They lack a generic methodological foundation that can be applied across diverse network domains. This fragmentation prevents the development of a standardized, scalable approach to DTN automation.

4) CONTEXT AND CONSISTENCY LIMITATIONS

LLMs are constrained by finite token limits within their context windows. As the session progresses, the accumulation of interaction history and network state data can cause the model to run out of tokens. This limitation makes it difficult to maintain a coherent view of the network's state over long operational cycles, often leading to the model being overwhelmed and truncating responses. Without a structured shared memory, a mechanism for reflection, and summarization strategies to preserve relevant context, autonomous agents struggle to maintain consistency and learn from past iterations.

B. DESIGN OBJECTIVES

The primary aim of this framework is to enable the end-to-end lifecycle management of DTNs, from inception to operation. To achieve this aim and overcome the identified challenges, we define the following core design objectives:

O1: End-to-End Closed-Loop Automation. The framework shall minimize human involvement by employing a multi-agent architecture. Specialized agents (e.g., Perception, Reflection, Planner, Coder, Tester, Controller) must

collaborate to autonomously drive the process from a high-level natural language use case to a fully operational DTN. Essentially, the system must operate in a continuous closed loop, perceiving the network state and adapting the design to ensure that the operational intent is persistently maintained.

O2: Trustworthiness via Iterative Automated Test-Driven Development and Sandboxing. To mitigate hallucination risks, the framework must enforce a strict “verify-before-deploy” paradigm. All artifacts must undergo rigorous validation in an isolated sandbox. Only code that compiles, passes all tests, successfully executes, and satisfies KPIs in the digital twin space is permitted to affect the physical network.

O3: KPI-Driven Source of Truth. The system must rely on quantifiable KPIs as the ultimate source of truth. Rather than relying on qualitative descriptions, such as “high” or “low”, the framework must extract and enforce precise numeric targets (e.g., latency ≤ 10 ms) to objectively validate system performance and guide the optimization process.

O4: Virtual-to-Real Gap Mitigation. Recognizing that virtual validations may not perfectly predict physical behavior, the framework must treat applying control changes as a tentative step rather than a final state. The framework must continuously monitor the physical network after actuation. If the deployed solution fails to meet KPIs in the real world, despite passing sandbox validation, the system must detect this discrepancy and automatically trigger a new design iteration to bridge the virtual-to-real gap.

O5: Adaptable, User-Centric and Practical Operation. The framework must be versatile within its target domain, capable of supporting a range of network use cases without significant changes to the framework’s architecture or code. It must be practical and cost-effective, addressing the operational needs of network engineers. It should abstract the complexity of prompt engineering and model interaction, providing a user-friendly graphical interface where operators focus on high-level intent and KPI refinement rather than low-level coding.

O6: Model-Agnostic Architecture and Loose Coupling. The framework must be loosely coupled from specific LLM providers to avoid vendor lock-in and ensure adaptability. It should support a pluggable adapter architecture that allows seamless integration with diverse providers (e.g., OpenAI, Google, self-hosted, Open-Source LLMs) and diverse models (e.g., GPT-5, Gemini 3 Pro, Llama 3). These adapters must abstract provider- and model-specific constraints, such as context window limits, reasoning capabilities, and API quotas.

O7: Robustness, Consistency, and Efficient Context Management. The framework must ensure consistent operation over long lifecycles. By leveraging a shared Single Source of Truth Memory (SSOT-M) to decouple system state from the agent’s context, and by employing summarization strategies, the system ensures that valid solutions are found reliably without overflowing model limits. Furthermore, the framework must be

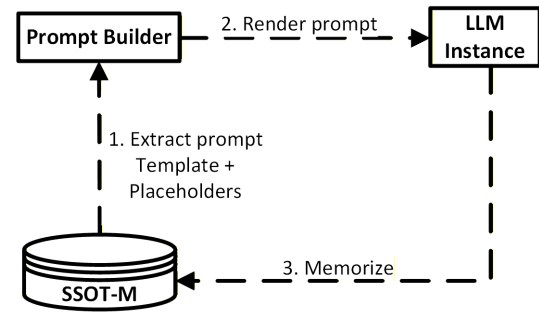


FIGURE 2. Single agent architecture.

resilient to truncated outputs and unexpected LLM results. It must implement robust error handling and retry mechanisms for transient issues, ensuring that the framework does not crash under any circumstances.

O8: Explainability and Auditability. The framework must provide transparent reasoning for its design decisions and actions. The system should generate human-readable explanations for the generated DTN modules and test cases, ensuring that operators can understand the outcomes when needed.

O9: Continuous Knowledge Base Enhancement. The framework should learn from past design iterations. It must accumulate executed strategies and failure modes to accelerate future optimizations and prevent the recurrence of known errors.

Based on these key design objectives, we propose the framework architecture in the next section.

IV. AGENTIC DTN FRAMEWORK ARCHITECTURE

In this section, we present the architecture of the proposed framework. Fig. 3 provides an overview of the end-to-end process. We first introduce the unified agent architecture, as well as the foundational Knowledge Base and infrastructure before detailing the operational lifecycle.

Single Agent Architecture. All agents within the framework adhere to a unified design pattern, as illustrated in Fig. 2. This architecture decouples the agent’s internal reasoning from the external logic. An Orchestrator acts as the driver: it triggers the *Prompt Builder*, which retrieves the necessary context (e.g., use case, KPIs, previous implementation attempts, and models) from the SSOT-M, which acts as a unified interface continuously aggregating runtime state (e.g., telemetry, reflection outcomes, designs, source code, and execution results), and the Knowledge Base, and renders a specific prompt. Fundamentally, all agents leverage a single LLM instance; however, they invoke it with different augmented prompts tailored to their specific roles and intents. That instance processes the received prompt to generate an artifact (e.g., a plan, code module, or test suite). The output is persisted, and based on the current system state and validation outcomes, the orchestrator determines the next step, such as parsing and executing the generated code. This design ensures that the agent remains a stateless reasoning engine,

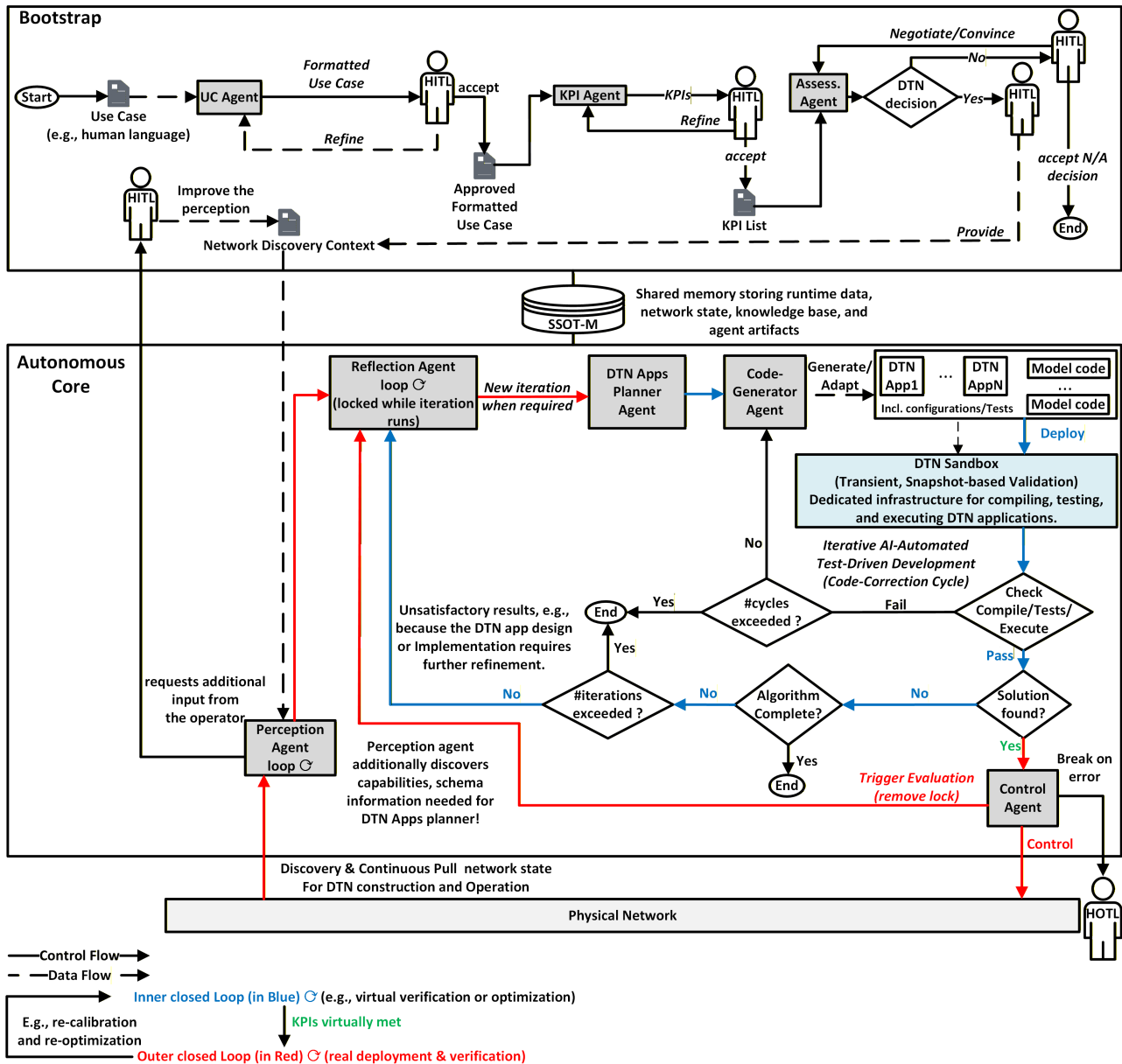


FIGURE 3. Overview of the Agentic DTN framework.

while the shared SSOT-M ensures consistency across the lifecycle.

Knowledge Base and Infrastructure. The framework leverages a knowledge base incorporating a model catalog, model representation methods (e.g., analytical such as graph, equation-based, rule-based; simulation; emulation; AI/ML; and (un-)structured data), and criteria guidelines such as design complexity, adaptability, scalability, computational and resource overhead, and fidelity, along with a scoring system [33]. It also integrates a target physical network and a dedicated DTN sandbox that provides compute, memory, and networking resources for compiling, testing, and executing DTN applications.

Operational Phases. The framework operation is divided into two main phases.

The *Bootstrap Phase* involves a human-in-the-loop (HITL) and covers use-case formulation, KPI specification, DTN applicability assessment, network discovery ingestion, and perception strategy approval. It ensures that the autonomous agents start with a validated and safe set of ground rules and objectives before taking control.

The *Autonomous Core* operates without human involvement and covers perception, reflection, DTN application planning, code generation, sandbox validation, and physical-network control.

Double Closed-Loop Control. We draw parallels with the ITU/IETF double closed-loop paradigm: an inner closed loop (depicted in blue) handles virtual verification and optimization. It is important to emphasize that this covers the complete design of the DTN applications including models,

implementation, testing, and validation, ensuring KPIs are virtually met before actuation. The outer closed loop (depicted in red) manages real-world actuation and verification, triggering re-calibration and re-optimization when KPIs in the real world are not yet met (e.g., due to the virtual-to-real gap) or are violated at a later point in time due to changing network conditions, for instance.

The proposed framework establishes a comprehensive lifecycle for Digital Twin Networks, transitioning from high-level human intent to autonomous operation. The process begins with the use case Formulation and Requirement Specification, where vague operational goals are refined into structured artifacts and measurable KPIs. Following a DTN Applicability Check and the ingestion of Network Discovery data, the Perception Agent establishes a continuous telemetry stream from the physical environment. The Reflection Agent continuously monitors this stream against the defined KPIs. Upon detecting a violation, control is passed to the autonomous planning and execution engine: the DTN Apps Planner Agent designs an optimization strategy, which the Code-Generator Agent implements and the Sandbox Validation suite rigorously tests in a safe, virtualized, and transient environment. This sandbox reflects the latest network snapshot based on the network state provided by the Perception Agent and is deliberately decoupled from live synchronization to strictly enforce verification before deployment. Only after successful validation does the Control Agent apply the changes to the physical network. This closes the loop, allowing the Reflection Agent to resume monitoring and ensuring that the network continuously adapts to maintain its performance targets. Ultimately, this framework unifies DTN design, implementation, testing, and operation, enabling network optimization in a near-zero touch manner. We gradually navigate the reader through the individual stages.

A. USE-CASE FORMULATION

The journey starts with a human stakeholder, such as a network operator, who submits a use-case description in natural language that captures high-level business or operational intents. Notably, the operator does not need to possess expert knowledge; a plain-language description of the desired use case is sufficient. This free-form description is first processed by the Use-Case Agent (UC Agent in Fig. 3), which transforms it into a more formalized and structured representation. Alternatively, if the operator already maintains a structured template or domain-specific format, this artifact can be injected directly. In both cases, the formalized specification is then reviewed by the human stakeholder, who can refine it over multiple iterations with the Use-Case Agent. Formalizing the use case is essential, because several downstream agents rely on this authoritative version stored in the SSOT-M. The formulation behavior is driven by the prompt shown in Fig. 4, which can be adapted or overridden by an operator-provided format specification when needed.

Here, the placeholder `raw_input` is substituted with the operator-provided text. The resulting structured bullets are

Use-Case Formulation Prompt

System role: You are an expert in network architecture.

Task: Extract metadata from the following use-case description. Present the output using a readable bullet-point format with the following structure:

- Title: ...
- Description, Assumptions, Scope: ...
- Domain, Lifecycle Stage: ...

Use Case: <raw_input>

FIGURE 4. Use-case formulation prompt.

TABLE 1. Example KPI list format.

ID	Name	Definition	Unit	Target
KPI-001	Uplink Throughput	Minimum sustained uplink rate per user towards the access network during busy hour.	Mbit/s	≥ 50
KPI-002	Service Availability	Fraction of time a critical application is reachable and operational for end users.	%	≥ 99.95
KPI-003	Energy per Bit	Average energy consumed in the access network to transmit one information bit.	J/bit	$\leq 1 \times 10^{-9}$

written into the SSOT-M as the approved use-case specification, enabling later agents to reference a stable artifact instead of the original free-form description.

B. REQUIREMENT SPECIFICATION

Once the use-case description has been formalized, the next phase derives a precise requirement specification. A dedicated Requirement Agent (KPI Agent in Fig. 3) consumes the approved use case from the SSOT-M and infers quantified metrics as KPIs. It is essential that requirements avoid vague, qualitative directions such as “high throughput” or “resilient network” and instead express measurable targets with specific bounds (e.g., minimum throughput in Mbit/s, maximum end-to-end latency in ms, or allowed packet loss in %). The human operator can then review this initial set of KPIs, remove unsuitable ones, introduce additional metrics, and adapt target values (e.g., tightening latency bounds or relaxing throughput thresholds). The finalized KPI set is captured in the SSOT-M and becomes the authoritative basis for all downstream agents. This is a fundamental step, as the resulting KPI list defines what the system optimizes for throughout the life cycle.

In practice, the KPI catalog can be stored as a simple tabular artifact so that both humans and agents can read and edit it. Each row defines a *KPI type*, which can give rise to many *KPI instances* in real deployments. For example, a latency KPI that specifies a maximum one-way delay of 30 ms per device will be instantiated once for every device in a network. Table 1 illustrates the structure used by the framework

KPI Generation Prompt

System role: You are a requirements engineer for Digital Twin Networks (DTNs).

Task: Generate a concise, strictly formatted set of quantifiable KPIs derived from the use case. These KPIs are the measurable targets the DTN will be designed to meet.

Output rules:

- Columns (exact order and names):
ID | Name | Definition | Unit | Target.
- IDs must be KPI-001..KPI-XXX, unique and sequential starting at KPI-001.
- Targets must be numeric with comparator when applicable ($\leq$, $\geq$, $<$, $>$, $=$).
- Avoid vague terms (e.g., “high”, “adequate”); every KPI must be directly measurable.
- Choose metrics relevant to the use case (e.g., latency, throughput, availability, loss, jitter, failover time, detection accuracy, energy per bit, cost per device) as applicable.
- No captions, no preceding or trailing text, and no duplicate or trailing column separators beyond the standard form.

Use Case: <use_case>

FIGURE 5. Requirements/KPI generation prompt.

DTN Applicability Prompt (summarized excerpt)

System role: You are a senior network architect.

Task: You are given a use case and its derived system requirements. Your task is to determine whether this use case is applicable for a Digital Twin Network setting.

DTN characteristics: Repeatability, Reproducibility, Real-time interaction, and Closed-loop automation.

Input parameters: <use_case>, <KPIs>, <human_feedback> (optional).

Question: Is the described use case a **valid candidate** for a Digital Twin Network setting? Answer with “Yes” or “No”. If “No”, justify why.

FIGURE 6. DTN applicability prompt.

together with three example KPIs from different domains. The Requirement Agent uses the prompt shown in Fig. 5.

C. DTN APPLICABILITY CHECK

With the use-case description and KPI catalog available, a DTN Applicability Agent (Assess. Agent in Fig. 3) is queried with both artifacts using the simplified instruction shown in Fig. 6 and requests a binary verdict (“applicable”/“not applicable”). If the model concludes that a DTN cannot reasonably adapt the network to meet the stated KPIs, it must articulate why, so that the operator can either accept the decision or challenge the agent’s reasoning in a negotiation loop. Once the agent finally accepts the DTN as a potential solution, the next stage of the framework is triggered and the workflow continues.

A positive answer indicates that the configured LLM reasoned that the DTN is a plausible option, not that it is the only or even the best approach. The operator still decides whether to invest in a DTN strategy or to pursue alternative tooling such as traditional simulators. Conversely, a final negative verdict halts the workflow before any planning or code generation takes place, saving effort when DTNs are unlikely to deliver the desired benefits.

D. NETWORK DISCOVERY CONTEXT AND BOOTSTRAP INPUTS

Once DTN applicability is confirmed, the orchestrator pauses until the operator provides the “Network Discovery Context”, so that the framework never infers infrastructure entry points or credentials but instead proceeds only after the human operator provides authoritative material describing the target physical network.

The discovery bundle can include artifacts such as OpenAPI [35] specifications, device endpoint addresses, credentials, and existing network documentation, as well as optional human-authored notes that capture operator-specific knowledge, for example, known constraints or preferred actuation hooks. During ingestion, the orchestrator normalizes all provided material into the SSOT-M, establishing a single, consistent reference that downstream agents consume directly.

In essence, this input defines how the DTN can interact with the physical network. Management-plane interfaces, exported schemas, operator playbooks, and device-specific actuation commands enable subsequent agents to generate artifacts specific to the physical network.

E. PERCEPTION AGENT

With the discovery bundle archived in the SSOT-M, the orchestrator invokes the Perception Agent. This agent consumes the formalized use case, KPI catalog, and curated discovery context to synthesize the continuous sensing script that feeds live telemetry and discovered metadata back into the SSOT-M. It also discovers capabilities and schema fragments (e.g., API schema), so that downstream agents inherit precise, machine-readable knowledge. Fig. 7 presents the shortened prompt supplied to the agent.

As soon as the perception code is generated and launched, the Graphical User Interface (GUI) shows the resulting telemetry stream in tabular form, so the operator can validate that the correct measurements are in place. When the agent cannot complete perception, for example because a referenced interface is missing or authentication fails, the orchestrator requests additional input from the operator. Follow-up inputs can include, for instance, a higher data-pull frequency for latency-sensitive KPIs (e.g., changing from 10 s to 1 s sampling), or explicit restrictions. Each such correction triggers a new perception-agent call, which regenerates or patches the sensing script to honor the updated intents. Once the human approves a satisfactory perception run, this marks the final direct intervention by a human. The sensing script continues running in parallel throughout the entire DTN

Perception Agent Prompt (summarized excerpt)

- **Role and intent:** Act as the Perception Agent for the physical space, emitting exactly one runnable script.
- **Input parameters:** <Use Case>, <KPI List>, <Network Discovery Context>, <Previous Attempts> (incl. code output, errors, produced raw data).
- **Artifacts to maintain:** Continuous sensing outputs & indexed snapshots, and structured network element and topology inventories.
- **Operating loop:** Perform a bootstrap phase to verify measurement tools, then enter a continuous sensing loop with measured delays, bounded retries, IPv4/IPv6 reachability checks, and KPI-specific measurement plans derived strictly from the provided context.
- **Safety guarantees:** Do not alter switches or topology; limit actions to approved hosts/containers, log every remote command with exit codes, and treat the log file as the single source of truth for reviewers.
- **Output contract:** Return one fenced code block that begins with “# file: ...” and exit with a non-zero status if any approved KPI lacks a specific measurement mechanism.

FIGURE 7. Perception agent prompt (summarized excerpt).

lifecycle to provide fresh evidence, while the orchestrator immediately advances to subsequent stages. The perception provides a complete view of the physical network, including the network element and topology information needed by downstream agents, and caches them in the SSOT-M. **From that point onward, the human-in-the-loop retires, and the framework advances autonomously** unless a subsequent physical-network actuation problem escalates control back to the operator (as Human-on-the-Loop (HOTL)).

F. REFLECTION AGENT

The Reflection Agent ingests the approved KPI list together with the live perception feed and determines whether the physical network satisfies each declared target. While the Perception Agent continuously acquires raw telemetry data, the Reflection Agent evaluates KPIs against the specified objectives. Every row copied from Perception stream gains a “fulfilled” label that records the binary verdict for that KPI instance. The evaluation proceeds cycle-by-cycle. For example, once the agent writes a latency measurement for “Device A”, it watches the perception stream until the next latency sample for the same device arrives and then finalizes the verdict for that cycle before moving on. Fig. 8 summarizes the prompt used to guide the agent’s reasoning and code synthesis.

Whenever the Reflection Agent records a single KPI violation, the orchestrator immediately diverts control to the next agents: DTN Apps Planning synthesizes the design strategy, Code-Generator produces the candidate implementation and tests, the sandbox validation suite exercises them, and the Control Agent enforces the approved change set on the

Reflection Agent Prompt (simplified excerpt)

- **Role:** Produce a ready-to-run script that evaluates KPI fulfillment without triple-quoted literals or auxiliary prose.
- **Input parameters:** <KPI List>, <Perception Stream (Raw Data)>
- **Data ingestion:** Load The perception stream, trim whitespace, deduplicate by (KPI_ID, description) keeping the freshest measurement, and reuse the mirrored KPI list directly for targets.
- **Evaluation logic:** Parse KPI target operators, compare them to the raw values, and mark each row as fulfilled or unfulfilled; raise a hard exception if any KPI cannot be decided deterministically.
- **Output contract:** Return one fenced code block that writes stream outputs for each reflection run and creates a dedicated snapshot file so that every execution leaves a traceable artifact.

FIGURE 8. Reflection agent prompt (simplified excerpt).

physical network. The reflection process continues to consume telemetry in the background, but it defers launching another decision cycle until the iteration completes and hands control back, ensuring that multiple concurrent optimization plans do not overlap or race.

G. DTN APPS PLANNER AGENT

The DTN Apps Planner Agent translates the approved use case, structured KPI catalog, network layout, reflection snapshots, and curated knowledge base (model catalog, representation methods, and the selection criteria introduced in [33]) into a runnable DTN application portfolio. In doing so, it effectively *designs the DTN*: it decides which DTN applications are required, which KPI gaps or lifecycle phases each application addresses, which DTN models each application should employ, and how these models are interconnected. For every application, the planner selects models together with appropriate modeling techniques (e.g., graph-based models, simulation, emulation, AI-based approaches, or combinations thereof) and specifies how models expose and consume data so that the overall DTN forms a coherent, modular architecture. It prioritizes reuse and extension of existing models over creating new ones and emits a specification that the downstream code-generation agent can implement directly. Fig. 9 provides a simplified excerpt of the planner prompt.

H. CODE-GENERATOR AGENT

The Code-Generator Agent translates the design into exactly two files: a self-contained module file containing the implementation, and a separate test suite file that enforces its contract. It acts as the system’s software factory, converting the abstract architectural plan into specific, executable artifacts. Each invocation is based on the DTN design, use-case, network layout, KPI list, validation history, a sandbox capability manifest (e.g., OS image, installed tools), and prior

DTN Apps Planner Prompt (simplified excerpt)

- **Role and intent:** Act as the DTN Apps Planner, producing a self-contained plan that downstream LLMs can implement without additional clarification.
- **Input parameters:** <Use Case>, <KPI List>, <Reflection Outcomes>, <Knowledge Base>, <Network Elements/Topology>, <Previous Design Plans>, <Previous Outcomes (code/test/validation logs)>
- **Portfolio rules:** Prioritize model reuse; for every application state whether a model is reused, extended, or new, and justify the choice via evaluation-criteria identifiers.
- **Execution guidance:** Describe inter-application data exchanges, stop/continue rules tied to KPI fulfillment, and hooks for selecting the strategy which lead to KPI fulfillment.
- **Output contract:** Produce the mandated section structure (Portfolio Overview, Application Blueprints, Cross-Application Dependencies, Formal Architecture diagram, Handoff Notes) using identifier-safe names and explicit API/configuration details.
- **Persistence expectations:** Assume the orchestrator versions every plan per iteration; do not reference external context beyond the provided artifacts.

Code-Generator Prompt (simplified excerpt)

- **Test-first mandate:** Generate test suites before implementation, fix/extend tests if needed, and ensure the final module matches the asserted behavior exactly.
- **Emulation discipline:** When emulation is selected, bootstrap the network to be emulated based on the exposed image information.
- **Persistence & observability:** Version artifacts, capture logs, track model-usage, and write solution artifact only after KPI-compliant evidence (this artifact acts as the marker for the next step to confirm a solution was found and identify which solution it is).
- **Validation:** Map prior test failures to targeted fixes, add tests that would catch each issue, and halt optimization once KPIs pass to avoid overlapping optimization cycles.
- **Input parameters:** <Use Case>, <KPI List>, <Network Elements/Topology>, <DTN Design>, <DTN Sandbox Image Capabilities>, <Artifacts/Validation Histories>.
- **Output contract:** Return structured document with `module_code`, `tests_code`, `planner/additional model usage lists`, a corrective summary tied to validation hints, and an `algorithm_complete` flag justified by search coverage.

FIGURE 9. DTN Apps planner prompt (simplified excerpt).

FIGURE 10. Code-generator prompt (simplified excerpt).

executor artifacts for historical validation context. Every iteration emits a versioned pair of files (module and test suite). The module code contains the complete implementation of the DTN applications, including the model instances and their interdependencies, as designed by the DTN Apps Planner Agent. The test suite encompasses tests that validate the module functionality. The agent is instructed to write tests before code, inspect previous failures occurred in the sandbox, and patch both artifacts. Fig. 10 captures the condensed enforcement rules.

I. SANDBOX VALIDATION AND ITERATIVE REFINEMENT

After the module and test suite have been generated, they will be compiled and subsequently executed. If any step fails (e.g., compilation error, crash in module or test, failing test), a validation summary is generated and passed back to the Code-Generator Agent for correction. This agent receives the current summary along with the history of previous attempts and generates new artifacts. This stage presents an **Iterative AI-Automated Test-Driven Development loop**. To prevent infinite loops, we restrict the number of code correction cycles; either a solution is found within this limit, or the process terminates. If a solution is successfully validated in the virtual environment, it is transmitted to the Control Agent for actuation in the physical network. If no solution is found, the framework checks the `algorithm_complete` flag provided by the Code-Generator Agent. This flag signals whether the executor can find a valid solution when one exists, based on the algorithm design, and thus whether

the agent has exhausted the theoretical search space for the given problem. If the flag is true, the process halts, as further attempts are deemed unlikely to succeed, notwithstanding potential gaps between the sandbox validation and the physical network. For example, strict latency KPIs might be consistently unmet in the sandbox due to virtualization overhead, even though the proposed solution would perform adequately on the target physical hardware. However, if the algorithm is deemed incomplete, the framework triggers a new iteration. The Reflection Agent is invoked to assess the unmet KPIs, subsequently activating the DTN Apps Planner to refine the application design. This iterative loop allows the system to address unsatisfactory results caused by initial design limitations. To ensure termination, the framework enforces a maximum bound on the number of allowed iterations.

J. CONTROL AGENT

The Control Agent serves as the bridge between the sandbox validation and the physical network. Once a solution artifact has been validated in the sandbox, this agent translates the actuation plan into specific, executable actuation logic targeting the production environment.

It consumes the validated solution artifact along with the authoritative network inventory and topology provided by the Perception Agent. It ensures that actuation commands use precise, discovered interface identifiers rather than heuristics. The agent generates a controller module designed to be idempotent and safe. This module handles low-level network

Control Agent Prompt (simplified excerpt)

- **Safety and Convergence:** The generated code must include pre-checks, post-checks, and explicit wait times for network convergence (e.g., ARP propagation after VIP moves).
- **Observability:** All actions are logged, and the script must emit a machine-readable status file confirming success or diagnosing failure.
- **Rollback Capability:** Generate a rollback plan alongside the actuation code to revert changes if the actuation fails.
- **Input parameters:** <Use Case>, <KPI List>, <Validated Solution Artifact>, <Network Elements/Topology>, <Network Discovery Context>, <Previous Actuation Attempts>.
- **Output contract:** Return structured document with `controller_code` and `header_json` containing filename and summary of the actuation approach.

FIGURE 11. Control agent prompt (simplified excerpt).

operations such as Virtual IP (VIP) address migration, route updates, or traffic shaping. The agent's simplified prompt, shown in Fig. 11, enforces strict operational discipline.

The framework executes this script, capturing standard output and error streams. If an error occurs, the framework immediately escalates to a human-on-the-loop, requiring manual intervention. Given the critical nature of this step, the agent does not attempt to reflect or retry autonomously; instead, it prioritizes restoring system stability and notifying the Human-on-the-Loop, who can decide the next step. Successful execution results are recorded in an actuation history log, effectively closing the control loop. This log serves as a critical audit trail, enabling operators to trace every automated change back to its originating intent. The Reflection Agent will automatically re-trigger the optimization loop whenever it detects an unfilled KPI.

V. SYSTEM IMPLEMENTATION

This section outlines the implemented framework. The system adopts a microservices-based architecture, as depicted in Fig. 12, which presents the high-level design and the interactions among its components while emphasizing clear separation of concerns. In the following, we detail each component to clarify its responsibilities and runtime interplay. It is important to note that while the architecture is adaptable, components, including the physical network testbed and the sandbox presented here mainly support network management and topology optimization use cases.

Physical Network: The physical network is emulated using Containernet [36]. This emulation runs on a dedicated **Physical Network Host**, where network nodes are instantiated as Docker containers to provide isolation.

Physical Network Service: A dedicated containerized service running on the **Management Host** that manages the physical network infrastructure. Since the emulated network

does not offer a unified API, we developed this service as an abstraction layer. It exposes an OpenAPI interface, allowing the sensing and controller agents to refer to it to explore, sense, and act on the real network.

DTN Sandbox: An isolated environment for compiling, executing, and validating generated DTN applications as well as their tests. This sandbox is hosted on a separate **Sandbox Host**. The Orchestrator executes generated code remotely on this host.

Single Source of Truth Memory (SSOT-M): A file-system-based storage layer that persists all artifacts, including use-case definitions, KPI lists, generated codes, network discovery context, as well as logs for perception, reflection, controller, and executors. This ensures that all agents operate on consistent, versioned data.

Orchestrator and Agentic Workflow: Running on the **Management Host**, the Orchestrator is implemented in Python using FastAPI [37] and LangGraph [38]. It coordinates the lifecycle by executing agents (Perception, Reflection, DTN Apps Planner, etc.) as distinct nodes within a directed graph and handling transitions based on their outputs and the overall system state. The underlying agentic framework is provider-agnostic, utilizing adapters to support models from OpenAI, Anthropic, and Google Gemini, as well as on-premise models.

Graphical User Interface (GUI): A React-based [39] web application running on the **Management Host** that enables operators to submit use cases, review KPIs, and check the initial perception mechanism. It also displays telemetry streams, reflection outcomes, and actuation logs, providing transparency into the DTN lifecycle.

We elaborate on these components in more detail in the following paragraphs.

A. PHYSICAL NETWORK EMULATION AND ABSTRACTION

We emulated the physical network using Containernet, which allows network hosts to be Docker containers. This design choice offers several advantages. Each network node (e.g., server, client) runs in its own container with a dedicated file system and resource limits, closely mimicking real-world hardware isolation. We can deploy actual application binaries and services (e.g., web servers, databases, custom Python scripts) inside the nodes. The framework supports the use of Open vSwitch provided by the underlying emulator or custom switches running as containers. The underlying infrastructure supports dynamic reconfiguration, allowing the Control Agent to modify network parameters (e.g., link bandwidth, latency, routing tables) at runtime to enforce optimization decisions. The physical network service exposes a REST API [40] with an OpenAPI interface that allows the Orchestrator and Agents to interact with the network programmatically. This includes capabilities to discover the network topology, pull telemetry data (for the Perception Agent), and execute actuation commands (for the Control Agent).

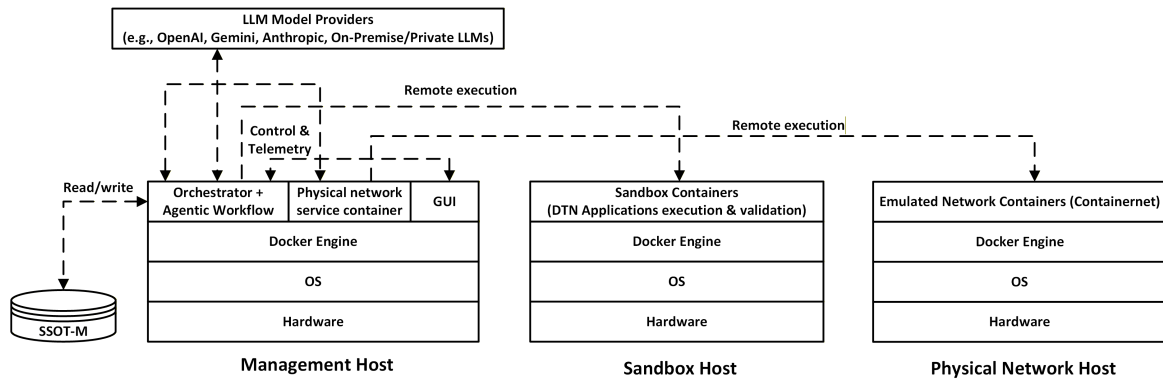


FIGURE 12. Distributed software architecture showing the separation of concerns across different DTN components.

B. DTN SANDBOX FOR EXECUTION AND VALIDATION

We established a DTN Sandbox as an integral subsystem of the framework. Located on a dedicated host, it provides a transient, isolated environment for compiling and validating generated DTN applications regardless of the modeling technique chosen by the Planner Agent (e.g., simulation, emulation, or analytical methods) and implemented by the Code-Generator Agent. This ensures that every proposed change is verified in a safe space that mirrors the physical network's state at the latest snapshot provided by the Perception Agent. Being decoupled from the live environment, it guarantees that virtual changes do not cause disruptions. The sandbox environment operates as a container whose image supports various software runtimes and toolchains (e.g., Containernet for emulation, Python environments for simulation, as well as necessary OS tools, libraries, and testing frameworks to support the AI-automated test-driven development process). For instance, if the plan mandates an emulation-based approach, the sandbox operates as a container capable of spawning sibling containers, functioning as the orchestration layer for the nested emulated network. A key configuration of the sandbox container is the bind mount of the host's Docker socket. This mount allows the sandboxed validation process to issue Docker commands against the host engine without running a full Docker daemon inside the container. At the same time, the mount avoids duplicating heavy virtualization layers and ensures that resource limits and isolation are still enforced by the host Docker engine. This pattern is well suited to scenarios where the DTN sandbox must dynamically orchestrate short-lived experimental Containernet nodes but should remain lightweight and easy to reproduce.

Conversely, if a simulation or analytical approach is selected, the corresponding runtimes and solvers must be available. To ensure adaptability, the Code Generator agent is capable of deriving missing dependencies during its bootstrap phase and generating code to install them, thereby dynamically tailoring the sandbox to the specific needs of the DTN application under test.

Overall, this sandbox approach ensures that if a generated artifact passes validation in the virtual environment, it has a

good probability of functioning correctly when deployed to the physical network.

C. ORCHESTRATOR AND AGENTIC WORKFLOW

The core logic is encapsulated in the orchestrator, which defines the lifecycle as a directed graph. The system transitions between states, such as *Perception*, *Reflection*, *Planning*, *Code Generation*, *Execution*, and *Control*, based on the outcomes of each step. For instance, if the Reflection Agent detects a KPI violation, the graph transitions to the Planner node. If the Sandbox Validation fails, the graph loops back to the Code Generator for refinement.

The framework intelligence resides in its agents, which are implemented as nodes in the LangGraph workflow. Each agent is designed with a specific role and a strictly defined interface. We employ a structured prompting strategy where each agent is given a "System Role" (e.g., "You are a Senior Network Architect") and a "Task Description". The prompts explicitly define the input artifacts (from the SSOT-M) and the required output format. It is important to note that agents do not maintain internal state between executions. Instead, they read the full context from the SSOT-M and write their results back to it. This evolving context provides a level of statefulness across the lifecycle.

The framework exposes settings to tailor the behavior of the agents and can be configured with different LLM providers. In this work, we executed the framework using three providers: OpenAI, Google Gemini, and Anthropic. In the case of Google Gemini and Anthropic, all agents utilize the same model. In the case of OpenAI, we distinguished between model generations. For GPT-5, we used a hybrid approach: we assigned `gpt-5` to the planning and reasoning agents (e.g., Use Case Formalizer, Requirement Generator, Applicability Check, Planner, Validation) to leverage its high-level reasoning capabilities, and `gpt-5-codex` for tasks requiring code generation (Perception, Reflection, DTN Code Generation, and Control). However, for the newer GPT-5.2, we adopted a unified strategy, employing the single `gpt-5.2` model across all agents, as no specialized Codex variant was available at the time of this work.

To ensure robust execution control, the framework allows the code generator a configurable number of code correction cycles to fix compilation, test, or sandbox execution issues within a single design proposal, and limits the global design refinement loop to a maximum number of iterations to prevent infinite loops. The code correction counter is reset at the beginning of each new iteration, allowing the code generator and sandbox testing a fresh set of attempts for every new design plan. Finally, regarding context and memory management, when the accumulated history (validation logs, artifacts, planner context) exceeds 150,000 tokens, it is summarized to retain essential context without overflowing the model's window. Furthermore, execution logs are truncated to 5000 characters per field to prevent verbose outputs from consuming excessive tokens. However, standard error is preserved, and high-level error stack traces are extracted and summarized to ensure the model retains visibility into critical failure causes.

D. SINGLE SOURCE OF TRUTH MEMORY (SSOT-M)

To ensure consistency across the agents, we implemented a file-system-based SSOT-M. The memory is organized by session ID, with dedicated subdirectories for each stage of the lifecycle. Every artifact is versioned which allows the system to track the running evolution and enables rollback capabilities. A central log records every state transition, agent decision, and human intervention, providing a complete audit trail of the DTN's operation.

E. GRAPHICAL USER INTERFACE (GUI)

To make the framework accessible to operators who do not directly interact with the underlying APIs or file system, we implemented a lightweight graphical user interface. The GUI is implemented as a React-based web application running on the Management Host and connects to the orchestrator via REST APIs.

F. SUMMARY

These components establish a robust workflow spanning emulated infrastructure, sandboxed testing, and centralized management. By enforcing clear interfaces, including REST APIs for network interaction, SSOT-M-backed artifacts for agent coordination, and a GUI for human oversight, the framework ensures that high-level design intents are consistently translated into validated actuations. This modular architecture provides the prerequisites for the experimental evaluation detailed in the next section.

VI. EVALUATION

In this section, we evaluate the framework through two distinct use cases: Multi-Access Edge Computing (MEC) Service Migration and Network Slicing. We execute these scenarios using a variety of LLMs from different providers, including Google Gemini, OpenAI, and Anthropic. Due to space constraints, we provide a detailed, step-by-step demonstration of the first use case, illustrating how the framework executes it and highlighting key decisions, while the second

use case is presented more concisely. To ensure statistical significance and derive robust insights, we conducted multiple runs for both use cases.

The experimental testbed consisted of three virtual machines. The Management Host and the Physical Network Host were each provisioned with 16 vCPUs and 16 GB of RAM, while the Sandbox Host was assigned 8 vCPUs and 16 GB of RAM. All hosts ran Ubuntu Linux (Kernel 5.15). It is important to note that the performance metrics reported in this section are primarily influenced by the inference overhead of the cloud-based LLM providers, rather than local processing power or network conditions.

A. USE CASE 1: MEC SERVICE MIGRATION

The first use case focuses on optimizing connectivity to a MEC server. This scenario is particularly relevant for 6G networks, where low-latency access to edge services is essential. Specifically, this use case aims to dynamically relocate compute services closer to users to maintain strict latency requirements. Seamless MEC migrations are therefore a well-suited evaluation use case for Digital Twin Networks: they require accurate situational awareness, automated corrective plans, rapid code synthesis, and safe actuation on heterogeneous infrastructure. By exercising migrations triggered by KPI violations, we show that the agentic framework closes the loop across every stage: use-case capture, requirements formalization, KPI assessment, perception, reflection, DTN design, code generation, sandbox testing, and final actuation on the physical network.

First, we present a single run using the Gemini-3-preview model to illustrate the framework operation. We then summarize the results of multiple runs across different models, including statistics. For demonstration purposes, we intentionally selected a run that needed multiple autonomous iterations and code correction cycles.

Before replaying the operational loop, we briefly describe the emulated physical network that the framework will optimize. We then replay one fully successful end-to-end session. We outline the chronological execution from the network-operator perspective and explicitly surface the artifacts each stage produced.

a: Physical Network Layout and Status-Quo

The evaluation runs on the Containernet-based emulated network shown in Fig. 13. Seven software switches (s1-s7) create a multi-hop mesh. Three user endpoints (Client1-Client3) attach to different access branches to capture heterogeneous latency paths, while six alternative MEC hosts (A-F) are available. The MEC service is initially active on the host "MEC". Each link carries an explicit one-way delay budget. All candidate MEC locations expose the same virtual IP (10.0.0.10) but retain unique MAC addresses, meaning only one host may actively advertise the service at any given time. Each switch terminates a dedicated MEC-facing port that can be activated when the workload migrates into its subtree, while "empty" MEC locations remain silent so that upstream switches and hosts do not relearn the

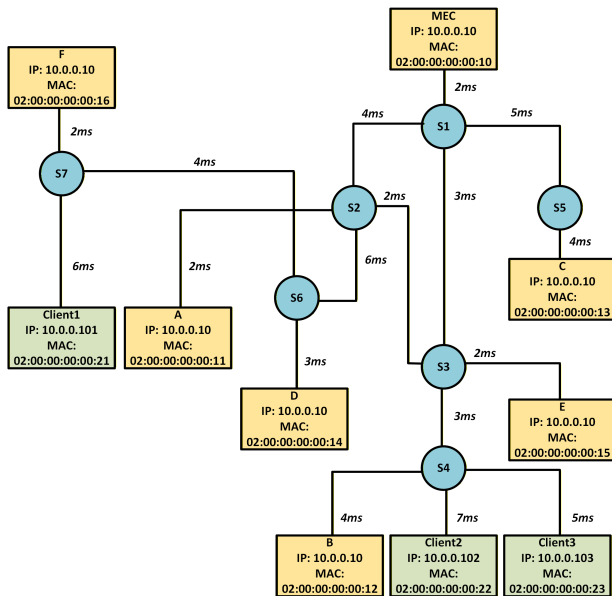


FIGURE 13. Emulated physical network for Use Case 1: MEC service migration.

Baseline ping measurements before optimization

```

$ docker exec mn.Client1 ping -c 2 10.0.0.10
... time=44.576 ms
... time=44.581 ms

$ docker exec mn.Client2 ping -c 2 10.0.0.10
... time=30.416 ms
... time=30.454 ms

$ docker exec mn.Client3 ping -c 2 10.0.0.10
... time=26.456 ms
... time=26.408 ms

```

FIGURE 14. Baseline ping evidence before DTN optimization.

ARP mapping prematurely. Spanning Tree Protocol (STP) is enabled to keep the multi-switch mesh loop-free, and the Containernet autoStaticArp helper is disabled so that ARP learning happens dynamically during migrations. As a result, the framework performs migrations entirely at Layer 2.

Before launching the DTN workflow, we captured baseline ICMP latency from each client to the current MEC host. These samples, summarized in Fig. 14, mirror the operator's "status quo" visibility.

Motivated by the elevated RTT of Client1, the operator explicitly triggers the DTN framework to optimize the network, limiting the closed-loop search to RTT improvements. While the framework can target multi-objective optimization, this demonstration pins the objective vector to RTT so the narrative stays focused on latency control.

LLM-Formalized Use Case

Title: Reliable Low-Latency Client-to-MEC Connectivity with Server Migration

Description: Ensure the network consistently delivers reliable, low-latency connectivity between Client1, Client2, and Client3 and a MEC server that may be migrated among hosts A, B, C, D, E, or F, so business-critical applications continue operating seamlessly under real-world load, fault, and mobility conditions.

Assumptions: All clients have IP reachability to every candidate MEC host (A-F) within the administrative domain. MEC workloads may be live-migrated or re-instantiated with session continuity (via VIP/anycast/service registry and fast re-resolution). The underlay provides redundant paths and fast convergence/failover to avoid noticeable service interruption. Latency-sensitive QoS policies are in place, with reserved capacity and headroom for bursty traffic. East-west latency between hosts A-F is low and predictable; inter-host links are not the bottleneck. Security policies (e.g., segmentation, encryption) persist across migration targets without manual reconfiguration. Observability exists with defined SLOs/SLA metrics (latency, jitter, loss, availability) and alerts tied to them. Orchestration/automation can trigger and complete migrations with minimal impact and propagate network changes rapidly (routing/DNS/service discovery). Required non-network prerequisites for migration (e.g., time sync, shared storage/images) are satisfied.

Scope: Domain: Multi-access Edge Computing (MEC) networking and service mobility; Lifecycle Stage: Design and Operations (assurance/resilience)

FIGURE 15. LLM-formalized use-case artifact stored in the SSOT-M.

1) EXECUTION WALKTHROUGH a: USE-CASE NARRATIVE SUBMISSION

The operator began by submitting the use-case narrative in the GUI. The following text was submitted:

"Ensure that the network infrastructure consistently provides reliable, low-latency connectivity between all clients (Client1, Client2, and Client3) and the MEC server, which can be migrated to another host (e.g., A, B, C, D, E, or F), allowing business-critical applications to function seamlessly under real-world conditions."

The first Use-Case Agent (UC Agent in Fig. 3) invocation formalized the free-form description provided by the operator into the structured representation shown in Fig. 15 so that downstream agents consume a well-defined artifact. The operator could have edited, added or removed any information of the use-case artifact at this stage. However, the operator decided to keep it as it is.

b: KPI-GENERATION AND APPROVAL

Once the operator approved the use-case artifact, the next step was to generate a structured KPI catalog using the Requirement Agent (KPI Agent in Fig. 3), capturing measurable targets derived from the use-case narrative. The outcome was a comprehensive list of 16 KPIs covering different aspects,

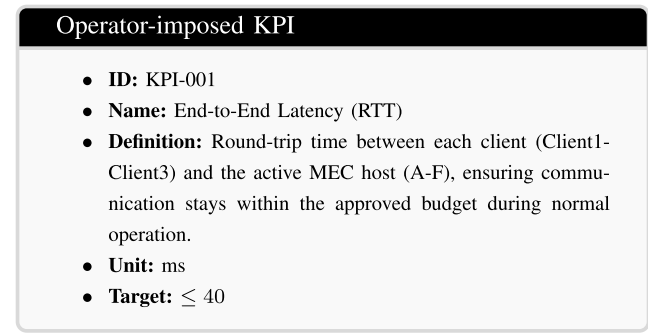


FIGURE 16. Operator-imposed KPI used for this evaluation.

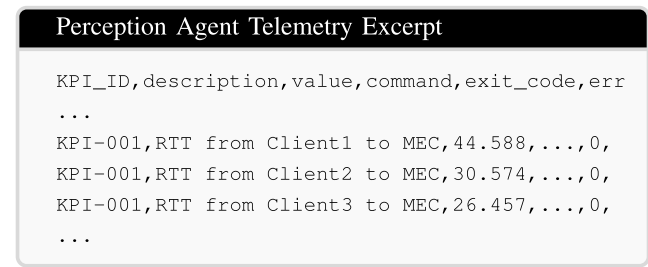


FIGURE 17. Perception-agent log snippet showing the KPI evidence used for approval.

including latency, bandwidth, packet loss, and security. The operator overwrote this list with a single KPI that captures the RTT every client must respect, as summarized in Fig. 16.

c: DTN APPLICABILITY CHECK

With the KPIs approved, the DTN Applicability Agent evaluated the scenario and confirmed that a Digital Twin Network is the appropriate mechanism to achieve these targets.

d: NETWORK DISCOVERY / BOOTSTRAP INPUT

Next, the operator contributed a specific discovery bundle so that the downstream Perception agents work against authoritative evidence. For this session the operator relied entirely on the OpenAPI specification exposed by the physical-network abstraction service. This input describes every sensing and actuation endpoint, and the full schema for request and response payloads. Although the framework allows additional free-text annotations (e.g., situational hints or temporary policy flags), the operator did not provide any supplemental information beyond the OpenAPI artifact.

e: PERCEPTION AGENT EXECUTION

The Perception Agent synthesized a self-contained Python script that calls the sensing endpoints exactly as published by the network-abstraction service. The script issued the prescribed ICMP measurements, and stored every row in the SSOT-M, ensuring later agents never have to infer how metrics were captured. Although the framework allows free-text feedback revisions for the LLM to adjust the perception mechanism, such as modifying sampling frequency or adding extra probes, the operator approved the initial configuration after inspecting the live feed on the GUI, as shown in Fig. 17.

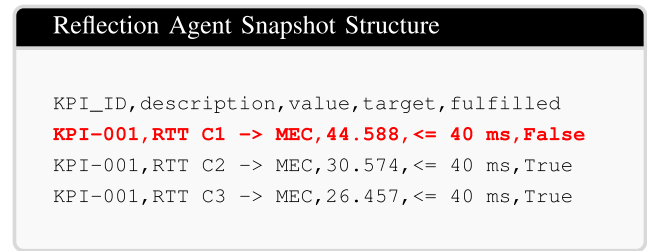


FIGURE 18. Reflection snapshot persisted in the SSOT-M with fulfillment verdicts.

These samples confirmed that all KPI instances (the three RTT measurements spawned from single KPI Type: KPI-001) were backed by empirical evidence, demonstrating a complete perception cycle. Once the operator approved this feed, the perception script continued running indefinitely in the background while the orchestrator triggered the Reflection Agent. At this point, the operator retired, and the framework continued running autonomously.

f: REFLECTION AGENT EXECUTION

Once perception settled into its continuous loop, the Reflection Agent waited for the first full perception cycle to complete so that a measurement existed for every KPI instance and only then evaluated the stream against the declared targets. Each reflection pass produced a CSV snapshot persisted in the SSOT-M with the structure illustrated in Fig. 18, which includes an explicit fulfillment flag per KPI instance. In this snapshot, Client1 violated the 40 ms bound, so the orchestrator immediately triggered the DTN optimization branch.

g: DTN CORE AUTONOMOUS EXECUTION

The framework required two autonomous iterations to converge. The first iteration, which spanned four code-correction cycles, ultimately failed: the first three code-correction cycles were rejected during unit testing due to logic errors. The fourth cycle passed unit testing but failed during the simulation phase because the functional logic included a bug in the code and test. As the generated code was flagged as algorithmically incomplete by the Code Generator Agent during the generation phase, the framework triggered a second iteration. For this new iteration, the DTN Apps Planner was initialized with a summarized history of the previous attempt, containing the failed plan, execution outputs, and specific error warnings. Leveraging this context to avoid repeating the same issues, the planner proposed a refined design based on simulation-based workload placement optimization. This design comprised a portfolio of three interconnected applications, where the planner successfully leveraged the internal knowledge base we curated, to instantiate specific submodels for each task. Table 2 details the specific models selected for each application, their reuse strategy, and the rationale behind these choices, as it was presented by the DTN Planner Agent.

TABLE 2. Models and reuse strategy for the generated applications.

Application	Model ID	Role	Reuse Strategy	Method	Rationale
Topology_Builder	Graph_Topo	In-memory weighted graph of switches, hosts, and links	Instance of Model: “Network Topology”	Graphs	Design Complexity (3), Fidelity (3): Standard method for static path analysis
	Node_Map	Categorizes nodes (Clients, Hosts, Switches) for lookup	Instance of Model: “Network Element”	Structured Data	Adaptability (3): Decouples node role from topology
Latency_Calculator	Dijkstra	Computes minimum cost path between two nodes	Instance of Model: “Traffic Analysis”	Math. Equations	Computational Needs (3): Efficient and exact for deterministic links
Migration_Solver	Optimizer	Comparison logic to select the best host	Instance of Model: “Resilience Optimization”	Optimization Problems	Fidelity (3): Guarantees mathematical best fit

The Topology Builder acted as the foundation by instantiating “Graph_Topo” from the “Network Topology” catalog. It used a graph-based representation to construct an in-memory weighted graph of the network derived from discovery data. Additionally, it deployed “Node_Map” from the “Network Element” catalog as a structured data model to categorize nodes (Clients, Hosts, Switches) for efficient lookup, effectively decoupling node roles from the topology structure. The Latency Calculator was responsible for predicting the RTT by instantiating “Dijkstra” from the “Traffic Analysis” catalog. This model employed mathematical equations (specifically Dijkstra’s algorithm) to compute the minimum cost path between nodes. It summed deterministic link delays to estimate the RTT ($RTT = \sum(Link_Delays) \times 2$). Finally, the Migration Solver functioned as the decision engine by instantiating “Optimizer” from the “Resilience Optimization” catalog. Modeled as an optimization problem using an exhaustive search strategy, it iterated through the finite search space of candidate hosts (A-F). Instead of structurally moving the MEC node, it simulated the migration by calculating the latency from each client to the candidate host to identify the optimal host that satisfies the 40 ms target for all clients.

The planner explicitly defined the cross-application dependencies: the Solver drives the control flow, while the Latency Calculator consumes the graph state provided by the Topology Builder. This modular design separated the state (Topology), the physics (Latency), and the logic (Solver), allowing the Code Generator to implement each component with clear boundaries. Based on this blueprint, the Code-Generator Agent synthesized the DTN application code and the corresponding test suite. It then entered the sandbox validation loop, where it compiled and executed the tests. During this second iteration, the Code Generator produced a valid solution on the first attempt, passing all validation checks. This confirmed not only that the code was syntactically correct and passed unit tests, but also that it executed successfully within the sandbox environment and identified a valid configuration that satisfied all KPIs in the virtual domain.

Perception Stream during Migration

```
KPI_ID,description,value,command,exit_code,err
KPI-001,RTT C1 -> MEC,44.431,...,0,
KPI-001,RTT C2 -> MEC,30.400,...,0,
KPI-001,RTT C3 -> MEC,26.418,...,0,
...
KPI-001,RTT C2 -> MEC,...,1,cmd failed
KPI-001,RTT C3 -> MEC,1090.744,...,
KPI-001,RTT C3 -> MEC,34.590,...,
KPI-001,RTT C1 -> MEC,...,1,cmd failed
KPI-001,RTT C2 -> MEC,52.318,...,
...
KPI-001,RTT C1 -> MEC,36.563,...,
KPI-001,RTT C2 -> MEC,38.577,...,
KPI-001,RTT C3 -> MEC,34.628,...,
```

FIGURE 19. Perception stream capturing the migration transient.

With the solution successfully validated in the sandbox, the framework initiated the handover to the physical domain. The Control Agent was triggered and provided with the validated solution artifact and the Network Discovery Context, which included the network element inventory, topology details, and the OpenAPI definition of the physical network service. Armed with this context, the Control Agent generated and executed the actuation code, successfully migrating the MEC service to a different target, Node A. As illustrated in Fig. 19, the perception stream captured a brief service disruption during the migration process, characterized by temporary packet loss and latency spikes. However, the network quickly converged, and subsequent measurements confirmed that all KPIs were successfully fulfilled, with Client1’s latency dropping below the 40 ms threshold.

2) COMPARATIVE EVALUATION

To assess the reliability and variability of the framework and to ensure statistical significance of the results, we

conducted an evaluation comprising 30 distinct execution runs for each of the three state-of-the-art LLM models, resulting in a total of 90 independent experiments. The models evaluated were OpenAI (gpt-5/gpt-5-codex and gpt-5.2), Google (gemini-3-pro-preview), and Anthropic (claude-opus-4.5).

To ensure a consistent starting point and isolate the performance of the autonomous DTN agents, we established a “frozen baseline snapshot.” This snapshot captures the system state after the human-in-the-loop phase concludes, containing the formalized Use-Case definition, the finalized KPI catalog, and the generated and validated Perception and Reflection agent code.

For each of these experiments, the orchestrator instantiated a new session based on this baseline template. This approach allowed every run to bypass the initial manual approval steps and proceed directly to the Post-Reflection stage. Upon detecting the predetermined KPI violation (Client1 latency), the framework automatically triggered the autonomous DTN optimization path (Planner → Code Generator → Sandbox Validation → Control), thereby testing the agents’ ability to independently resolve the issue under identical initial conditions. Since the framework is designed to run continuously to monitor for violations, we introduced a mechanism to terminate the session immediately after a solution is found and the migration is triggered.

To provide a comprehensive assessment of the autonomous framework, we analyze the results across multiple dimensions, including operational reliability, decision output, and execution time performance. Specifically, we measure the **Success Rate** (percentage of runs finding a solution), **Convergence Speed** (number of iterations/code correction cycles). Additionally, we analyze the **Host Distribution** to evaluate the determinism and variability of the decision-making process. We also evaluate the **Execution Time** for each involved agent to understand the performance overhead.

a: RESULTS FOR GEMINI-3-PRO

We conducted a focused evaluation of 30 consecutive runs using the Google Gemini-3-Pro-Preview model. The results demonstrated a 100% success rate (30/30) in finding a valid solution and successfully executing the migration. Remarkably, the framework required on average 1.03 iterations (Median= 1.0) to converge on a solution. The code generation was highly efficient, with an average of 2.13 correction cycles per session (Median=2.0).

Regarding the decision output, the planner exhibited a preference for two specific candidates: Node A (13 runs, 43.3%) and Node E (17 runs, 56.7%), both of which satisfy the latency constraints. In cases where Node E was chosen, the model explicitly aimed to minimize the average RTT among all clients, rather than merely fulfilling the individual KPIs. This variability in valid choices highlights the non-deterministic nature of the LLM’s search strategy within the solution space.

In terms of execution time, the framework demonstrated consistent performance across the 30 runs. The DTN Apps Planner required an average of 50.67 s (Median= 48.00 s, IQR=[46.75, 51.00] s) to design the optimization strategy. The Code Generator took an average of 169.40 s (Median= 140.50 s, IQR=[88.00, 215.00] s) to synthesize the implementation and test suite, while the Validation phase consumed 170.43 s on average (Median= 153.00 s, IQR=[84.00, 181.50] s). Finally, the Control Agent executed the physical network actuation in 98.10 s on average (Median= 98.00 s, IQR=[90.50, 107.00] s). These durations reflect the complexity of the tasks, with code generation and validation being the most time-intensive due to the iterative test-driven development loop and the overhead of spinning up isolated sandbox environments.

b: RESULTS FOR OPENAI GPT-5 / GPT-5-CODEX

We evaluated the OpenAI models: GPT-5 (reasoning) and GPT-5-Codex (coding) across 30 runs. The framework achieved a success rate of 60.0% (18/30). The system required an average of 1.93 iterations (Median= 1.5) to find a solution. Code generation involved an average of 4.63 correction cycles per session (Median= 3.5). Regarding host selection, the planner strongly favored Node A, selecting it in 16 runs (88.9%), while Node E was chosen in only 2 runs (11.1%).

In terms of execution time, the DTN Apps Planner took an average of 280.90 s (Median= 232.00 s, IQR=[164.75, 371.00] s). The Code Generator required significantly more time, averaging 772.21 s (Median= 568.00 s, IQR=[337.75, 1105.00] s). The Validation phase consumed 386.71 s on average (Median= 270.50 s, IQR=[167.00, 584.25] s). Finally, the Control Agent executed the actuation in 85.44 s on average (Median= 74.50 s, IQR=[68.50, 111.50] s).

c: RESULTS FOR OPENAI GPT-5.2

We also evaluated the newer OpenAI GPT-5.2 model across 30 runs. The framework achieved a success rate of 53.3% (16/30). The system required an average of 1.83 iterations (Median= 1.0) to find a solution. Code generation involved a significantly higher average of 9.53 correction cycles per session (Median= 5.5). Regarding host selection, the planner strongly favored Node A, selecting it in 14 runs (87.5%), while Node F was chosen in 2 runs (12.5%).

In terms of execution time, the DTN Apps Planner took an average of 211.10 s (Median= 118.50 s). The Code Generator required substantial time, averaging 943.60 s (Median= 519.00 s). The Validation phase was also time-intensive, consuming 932.30 s on average (Median= 522.50 s). Finally, the Control Agent executed the actuation in 57.60 s on average (Median= 57.00 s).

d: RESULTS FOR ANTHROPIC CLAUDE-OPUS-4.5

We evaluated the Anthropic Claude-Opus-4.5 model across 30 runs. The framework achieved a high success rate of 96.7% (29/30). The system demonstrated excellent

TABLE 3. Summary of performance metrics across LLM providers for use Case 1.

Metric	Gemini-3-Pro	OpenAI GPT-5	OpenAI GPT-5.2	Anthropic Claude-Opus-4.5
Operational Reliability				
Success Rate	100% (30/30)	60.0% (18/30)	53.3% (16/30)	96.7% (29/30)
Avg. Iterations to Converge	1.03	1.93	1.83	1.03
Avg. Code Correction Cycles	2.13	4.63	9.53	1.70
Decision Output (Host Selection)				
Selected Node A	13 (43.3%)	16 (88.9%)	14 (87.5%)	26 (89.7%)
Selected Node E	17 (56.7%)	2 (11.1%)	0 (0.0%)	0 (0.0%)
Selected Node F	0 (0.0%)	0 (0.0%)	2 (12.5%)	0 (0.0%)
Selected Node MEC	0 (0.0%)	0 (0.0%)	0 (0.0%)	3 (10.3%)
Execution Time Performance (seconds)				
Avg. Planner Time	50.67	280.90	211.10	148.07
Avg. Code Generator Time	169.40	772.21	943.60	437.23
Avg. Validation Time	170.43	386.71	932.30	144.00
Avg. Control Agent Time	98.10	85.44	57.60	83.04
Total Average Time	488.60	1525.26	2144.60	812.34

convergence, requiring an average of 1.03 iterations (Median= 1.0) to find a solution. Code generation was particularly efficient, with an average of 1.70 correction cycles per session (Median= 1.0), outperforming both Gemini and OpenAI in this metric.

Regarding host selection, the planner showed a strong preference for Node A, selecting it in 26 runs (89.7%). However, in a small minority of 3 runs (10.3%), the agent selected “Node MEC”, effectively deciding to remain on the current node, which failed to resolve the issue as the current node was already violating the KPI.

In terms of execution time, the DTN Apps Planner took an average of 148.07 s (Median= 140.00 s, IQR=[133.00, 152.25] s). The Code Generator required an average of 437.23 s (Median= 273.50 s, IQR=[208.75, 454.25] s). The Validation phase consumed 144.00 s on average (Median= 103.50 s, IQR=[75.00, 150.00] s). Finally, the Control Agent executed the actuation in 83.04 s on average (Median= 79.50 s, IQR=[70.50, 93.75] s).

e: RESULT SUMMARY

The comparative analysis, summarized in Table 3, reveals distinct performance profiles for each model. Gemini-3-Pro demonstrated the highest reliability and speed, achieving a 100% success rate with the lowest execution times across most phases. Its decision output exhibited significant variability, frequently selecting Node E. This choice was driven by the model’s self-imposed goal to minimize the average RTT across all clients, which was not part of the specified requirements. OpenAI’s GPT-5 showed strong reasoning capabilities in host selection (favoring the Node A) but suffered from lower success rates and significantly higher execution times, primarily due to code generation struggles. The newer GPT-5.2 model exhibited similar reasoning patterns but faced

even greater challenges in code generation and validation stability, leading to the highest execution times and a lower success rate. Additionally, it occasionally selected Node F based on hallucinated simulation assumptions. Anthropic’s Claude-Opus-4.5 achieved a high success rate; however, it wrongly selected the current MEC node three times.

B. USE CASE 2: NETWORK SLICING

In this scenario, we demonstrate the framework ability to enforce strict security policies and logical segmentation on a shared physical infrastructure. Unlike the first use case, which focused on performance optimization (latency), this case prioritizes security and isolation.

a: Physical Network Layout

The physical network topology, illustrated in Fig. 20, consists of a structure with 7 switches (s1-s7) and 6 clients (Client1-Client6). This shared infrastructure initially allows unrestricted communication between all endpoints.

1) USE-CASE FORMULATION

The operator provided the following narrative to the framework:

“We have a shared physical infrastructure with six clients (Client1 through Client6) connected to the same network, which is composed of Open vSwitch (OVS) switches. We need to enforce strict logical isolation to create two network slices. The first network should consist of Client1, Client3, and Client5. The second network should include Client2, Client4, and Client6. Traffic must be allowed between clients within the same group, but completely blocked between the two groups.”

This request challenges the framework to translate high-level intent (“logical isolation”) into low-level network configurations without disrupting intra-group connectivity. It

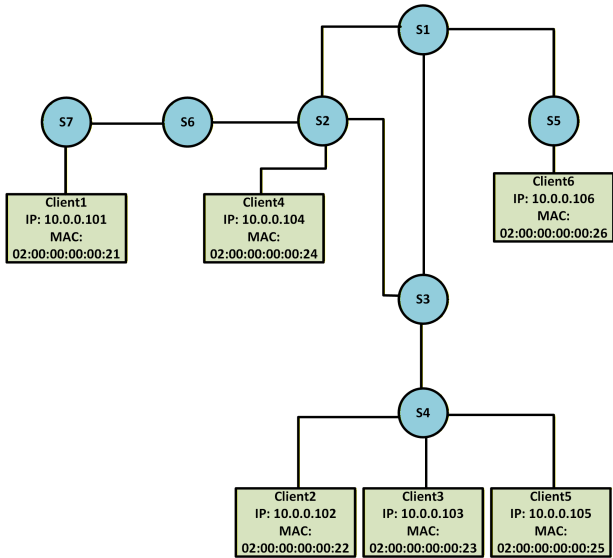


FIGURE 20. Emulated physical network for Use Case 2: Network slicing.

Generated KPIs for Network Isolation

- **ID:** KPI-002
- **Name:** Intra-Tenant Connectivity
- **Definition:** Verify that clients within the same tenant group (e.g., Client1 ↔ Client3) can successfully exchange packets.
- **Unit:** Reachability (Boolean)
- **Target:** True

- **ID:** KPI-003
- **Name:** Inter-Tenant Isolation
- **Definition:** Verify that traffic between clients in different tenant groups (e.g., Client1 ↔ Client2) is blocked.
- **Unit:** Reachability (Boolean)
- **Target:** False

FIGURE 21. KPIs for validating logical isolation.

is important to note that we did not provide any specific instructions on how to achieve this isolation, leaving the framework to autonomously determine the solution.

a: KPI-GENERATION AND APPROVAL

Based on the operator’s intent, the framework generated two distinct KPI types to validate both connectivity and isolation (see Fig. 21).

b: PERCEPTION AGENT EXECUTION

The Perception Agent generated a Python script to continuously monitor the network state. In this scenario, the agent instantiated a mesh of ICMP ping probes to verify reachability between all pairs of clients. This monitoring was essential to validate both the required intra-group connectivity and the mandated inter-group isolation. Fig. 22 shows a snippet of the initial perception log, confirming that the physical network

Perception Log: Initial Connectivity State

```
KPI_ID,description,value,command,exit_code
KPI-002,Intra-Group1: C1->C3,True,ping ...,0
KPI-002,Intra-Group1: C1->C5,True,ping ...,0
...
KPI-003,Inter-Group: C1->C2,True,ping ...,0
KPI-003,Inter-Group: C1->C4,True,ping ...,0
...
```

FIGURE 22. Perception log showing full network reachability before optimization.

Reflection Snapshot: Isolation Violation

```
KPI_ID,description,value,target,fulfilled
KPI-002,Intra-Group1: C1->C3,True,True,True
KPI-002,Intra-Group1: C1->C5,True,True,True
KPI-002,Intra-Group1: C3->C1,True,True,True
...
KPI-003,Inter-Group: C1->C2,True,False,False
KPI-003,Inter-Group: C1->C4,True,False,False
KPI-003,Inter-Group: C1->C6,True,False,False
...
```

FIGURE 23. Reflection snapshot showing fulfilled connectivity but violated isolation.

was fully reachable, which, while satisfying the connectivity KPIs, also indicated the lack of isolation.

c: REFLECTION AGENT EXECUTION

Following the perception phase, the Reflection Agent evaluated the network state against the defined KPIs. As the network was initially a shared flat domain with no restrictions, all clients could communicate with each other. Consequently, the intra-group connectivity requirements (KPI-002) were naturally satisfied. However, this unrestricted connectivity directly violated the inter-group isolation requirements (KPI-003), as traffic between tenant groups was permitted. Fig. 23 illustrates the reflection snapshot, highlighting the widespread violation of isolation policies which triggered the autonomous optimization process.

2) RESULTS OF THE AUTONOMOUS EXECUTION

We evaluated Use Case 2 using Google Gemini-3-Pro-Preview, OpenAI GPT-5 and GPT-5.2, and Anthropic Claude-Opus-4.5 models across 30 runs each. In addition to operational reliability metrics, we also measured the **Strategy Distribution** to analyze the different approaches the agents took to enforce isolation. The results are summarized in Table 4.

a: GEMINI-3-PRO

The framework achieved a 100% success rate (30/30). The successful runs converged rapidly, with an average of

TABLE 4. Performance metrics and strategy distribution for use Case 2.

Metric	Gemini-3-Pro	OpenAI GPT-5	OpenAI GPT-5.2	Anthropic Claude-Opus-4.5
Operational Reliability				
Success Rate	100% (30/30)	80.0% (24/30)	30.0% (9/30)	86.7% (26/30)
Avg. Iterations	1.00	1.17	1.77	1.57
Avg. Code Correction Cycles	1.73	1.90	7.27	3.73
Avg. Planner Time	72.27 s	184.30 s	190.27 s	273.67 s
Avg. Code Gen. Time	284.17 s	483.03 s	770.07 s	1130.57 s
Avg. Validation Time	184.80 s	214.66 s	1116.23 s	722.87 s
Avg. Control Time	175.12 s	164.83 s	113.00 s	181.35 s
Total Average Time	716.36 s	1046.82 s	2189.57 s	2308.46 s
Strategy Distribution (Successful Runs)				
OpenFlow Rules (ovs-ofctl)	23 (76.7%)	8 (33.3%)	4 (44.4%)	25 (96.2%)
VLAN Tagging (802.1Q)	7 (23.3%)	16 (66.7%)	5 (55.6%)	1 (3.8%)

1.00 iteration and 1.73 code correction cycles. Interestingly, the agents autonomously discovered two distinct valid strategies to enforce the isolation: 23 runs employed OpenFlow rules to filter traffic at the flow level, while 7 runs configured VLAN tagging (IEEE 802.1Q) to separate the tenant groups at Layer 2. Both strategies successfully satisfied the isolation KPIs.

b: OPENAI GPT-5

The OpenAI models achieved a success rate of 80.0% (24/30). The system required slightly more iterations on average (1.17) and code correction cycles (1.90) compared to Gemini. Notably, the strategy distribution was heavily skewed towards VLAN tagging, which was selected in 16 runs (66.7% of successful runs), while OpenFlow rules were used in 8 runs (33.3%). This indicates a strong preference in the OpenAI model for Layer 2 segmentation strategies.

c: OPENAI GPT-5.2

The newer OpenAI GPT-5.2 model achieved a success rate of 30.0% (9/30). The system required an average of 1.77 iterations and a significantly higher number of code correction cycles (7.27 on average). In terms of strategy, the successful runs were fairly balanced, with 5 runs (55.6%) employing VLAN tagging and 4 runs (44.4%) using OpenFlow rules. The high number of correction cycles and low success rate suggest that while the model can reason about both strategies, it struggles with the precise implementation details required for successful validation.

d: ANTHROPIC CLAUDE-OPUS-4.5

The Anthropic model achieved a success rate of 86.7% (26/30). However, it required a higher number of code correction cycles (3.73 on average) and iterations (1.57 on average). In terms of strategy, Claude-Opus-4.5 exhibited an overwhelming preference for OpenFlow rules, which were employed in 96.2% of the successful runs (25/26), with only a single run (3.8%) utilizing VLAN tagging. This suggests

a highly deterministic approach to solving the isolation problem compared to the other models.

e: RESULT SUMMARY

The comparative analysis reveals that while all models successfully generated valid isolation strategies, they exhibited distinct preferences. Gemini-3-Pro demonstrated superior reliability and efficiency, achieving a 100% success rate with minimal iterations. OpenAI GPT-5, while slightly less reliable (80%), showed a unique preference for Layer 2 VLAN segmentation, contrasting with the other models. The newer GPT-5.2 model struggled significantly with reliability (30% success rate) and required extensive code correction cycles, though it maintained a balanced approach between VLAN and OpenFlow strategies when it did succeed. Anthropic Claude-Opus-4.5 achieved a high success rate (86.7%) but required significantly more code correction cycles to stabilize its solutions, and it displayed a near-exclusive preference for OpenFlow-based enforcement. This divergence underscores that different LLMs may gravitate towards fundamentally different network engineering paradigms, even when presented with identical requirements.

C. ARCHITECTURAL ANALYSIS

An analysis of the results reveals multiple architectural advantages of the proposed agentic framework compared to rule-based and heuristic approaches, as well as non-agentic LLMs. Traditional network service deployment relies extensively on manual device configuration by network engineers, with service lead-times for new services reaching up to several months, dominated by the design and configuration phases [45]. Ericsson's industry analysis quantifies a beyond-5G network lifecycle at 54–72 weeks, broken down into requirements gathering (6–12 weeks), planning (12 weeks), design and simulation (12 weeks), fulfillment (12–24 weeks), and tuning (12 weeks) [46]. Testing of the SDN control plane alone can require many weeks of substantial manual effort [47]. The Bootstrap Phase of the proposed framework

TABLE 5. First-attempt (single-shot) vs. full agentic framework success rates (%) and improvement in percentage points (pp).

Model	UC1			UC2		
	1 st	Full	Δ pp	1 st	Full	Δ pp
Gemini-3-Pro	43.3	100.0	+56.7	43.3	100.0	+56.7
GPT-5	10.0	60.0	+50.0	36.7	80.0	+43.3
GPT-5.2	3.3	53.3	+50.0	13.3	30.0	+16.7
Claude-Opus-4.5	63.3	96.7	+33.4	6.7	86.7	+80.0

(see Fig. 3) can itself require significant time, as it involves human-in-the-loop activities such as KPI negotiations. However, in the traditional approach, the entire lifecycle is carried out manually. In contrast, once the Bootstrap Phase is complete, the Autonomous Core takes over and executes the full engineering cycle. As shown in Tables 3 and 4, this autonomous execution averaged approximately 8 minutes (Gemini 3-Pro, UC1) to 38 minutes (Claude-Opus-4.5, UC2), representing a reduction of multiple orders of magnitude compared to the engineering phases alone in the traditional lifecycle. While this comparison should be interpreted with care, since a comprehensive benchmark would require addressing the same baseline, the reduction remains substantial. Non-agentic LLMs, on the other hand, lack an inherent closed-loop feedback mechanism, which prevents reliable self-verification and iterative correction of generated outputs. To quantify this effect, we derive a *first-attempt success rate* from our experimental logs, defined as the proportion of runs where the initial code generation attempt in the first design iteration passed all validation checks without any correction cycle, which is equivalent to the outcome achievable by a non-agentic, single-shot LLM approach. As shown in Table 5, the closed-loop correction mechanism yields improvements ranging from +16.7 to +80.0 percentage points across all models and use cases.

The proposed framework operates at a higher level of abstraction, enabling the systematic translation of high-level requirements into effective system implementations. Despite this abstraction, the design ensures inherent explainability, where every stage (e.g., intent formulation, design, actuation) is fully traceable, providing a distinct advantage over “black-box” approaches. The execution traces of both use cases illustrate this process, showing how the framework autonomously navigated multiple design iterations and code correction cycles. When initial attempts failed validation within the isolated sandbox, the framework analyzed the results, reflected on the errors, and corrected the implementation.

Furthermore, it is important to acknowledge that the autonomous agents do not always converge on algorithmically optimal solutions. For instance, in Use Case 1, the generated solution relied on an exhaustive search strategy to identify the best migration host. While this approach is computationally inefficient for larger topologies, it was sufficient to satisfy the latency targets imposed by the operator. This highlights the KPI-oriented nature of the framework: the primary objective is to identify a valid

solution that satisfies all KPIs, rather than to derive a theoretically optimal algorithm. Conversely, if an operator were to impose a KPI that cannot be met by such an approach, the framework is expected to detect this limitation via the Reflection Agent. In response, it would trigger additional iterations to refine the design and implementation until a strategy capable of satisfying the stricter KPI is identified. Similarly, in Use Case 2, the autonomous derivation of security policies using conventional mechanisms (OpenFlow, VLANs) demonstrates the framework’s adaptability across diverse scopes without architectural changes. Here again, the operator was only required to communicate the intent, while the framework autonomously handled the entire remaining process.

Finally, leveraging multiple LLM providers demonstrated that, while performance profiles differ, robust orchestration is essential. This shows that reliability stems from the proposed framework architecture rather than specific model capabilities, enabling a rapid transition from design to physical actuation that would remain unattainable without this approach.

VII. DISCUSSION

This section discusses the broader implications of the proposed framework, analyzing its benefits and limitations, operational safety, and the lessons learned during its development and evaluation.

A. BENEFITS AND LIMITATIONS OF THE PROPOSED FRAMEWORK

A primary advantage of the proposed framework is its adaptable design, which is not tailored to a single network use case but rather provides a flexible architecture for diverse tasks. The evaluation demonstrates that the framework significantly accelerates the network optimization lifecycle. Tasks that traditionally require weeks or months of manual engineering across the entire lifecycle [45], [46], [47] are compressed into minutes. This automation not only reduces Operational Expenditure (OPEX) but also minimizes human error. Furthermore, the framework lowers the barrier to entry, as the operator does not need to possess expert knowledge; a plain-language description of the desired use case is sufficient to initiate the optimization process. However, we expect a cost shift from manual design, implementation, testing, and validation to more rigorous requirement definition and prompt engineering. A cornerstone of this reliability is the proposed Iterative AI-Automated Test-Driven Development methodology combined with history maintenance for reflection. By mandating that agents generate test suites before implementation and maintaining a history of past attempts to guide future iterations, the framework enforces a rigorous validation loop that actively counters LLM hallucinations. This approach not only ensures functional correctness but also accelerates convergence by providing precise feedback to the Code Generator when tests fail, drastically reducing the time spent on debugging compared to manual workflows.

A key strength of the framework is its generalization capability. As evidenced by the transition from Use Case 1 (Latency Optimization) to Use Case 2 (Network Slicing), the system adapted to a fundamentally different problem domain without any modification to its underlying code, architecture, or even agent prompts, validating the framework's adaptability within IP-based networks. While our current evaluation is focused on core network and topology management, the architecture itself is designed to scale to other domains such as radio access networks and mobility management. Incorporating these domains would primarily involve reconfiguring the framework, such as with enhanced knowledge base and specialized agent prompts, without requiring fundamental structural changes. Since the specific models are decoupled from the agentic architecture, leveraging next-generation models is a matter of configuration, allowing the system to immediately benefit from improvements in reasoning and code generation without system refactoring.

However, several limitations persist. The framework's performance is heavily dependent on external LLM providers. We observed occasional timeouts, internal server errors, and rate limits, which necessitated the implementation of robust retry mechanisms with exponential backoff. In addition, models occasionally provided malformed responses. While the framework caught these exceptions and was designed not to crash under any circumstances, handling these errors can potentially lead to extended running times. Furthermore, the inference latency of current state-of-the-art models can be significant, potentially limiting the framework's applicability in ultra-low-latency control loops. However, this is a general challenge in the field, and the DTN framework could mitigate this by caching optimization plans and executable code to be fetched at a later point in time for recurring scenarios. Additionally, the finite context window of LLMs poses a challenge for long-running sessions. Although we implemented summarization strategies to compress history, this compression inevitably leads to some loss of detail, which can affect the agent's reasoning in complex, multi-iteration scenarios. Finally, the use of public LLM services introduces privacy concerns regarding sensitive network data. This framework addresses this by supporting self-hosted and private models, allowing operators to make a trade-off between model performance and data privacy. Moreover, a hybrid approach can be employed where different agents utilize different LLMs, enabling sensitive tasks to be handled locally while leveraging the reasoning power of state-of-the-art public models for non-sensitive components. Finally, although the framework currently supports multi-modality only during the network discovery phase, for example, allowing the operator to provide extensive network documentation in PDF format, extending such support to the remaining agents was not necessary for this work. Nonetheless, future extensions may incorporate additional modalities, including images, audio, and video.

B. OPERATIONAL CONSIDERATIONS AND SAFETY ASPECTS

A critical operational challenge is the "virtual-to-real" gap. Solutions validated within the sandbox can fail to meet strict performance targets when deployed, due to physical network overheads not captured in the DTN models. This underscores the necessity of a high-fidelity sandbox environment and modeling. Emulations generally provide the accuracy required to prevent such failures. However, emulation itself is not infallible; discrepancies can still arise if the sandbox environment lacks sufficient resources to accurately replicate the timing and throughput characteristics of the physical network. In that case, a solution might fail to meet KPIs in the sandbox, even though it would have succeeded in the real world. The Reflection Agent in the presented framework continuously evaluates KPIs in the physical network, regardless of whether a solution was found, and triggers a new iteration if performance targets remain unmet. If the framework is unable to solve a problem after multiple design iterations or code correction cycles, this serves as a signal that the agent prompt or physical network API capabilities need refinement.

Regarding safety, the framework enforces a strict "verify-before-deploy" policy. No code touches the physical network without passing the sandbox validation. In the event of an actuation failure during the final deployment phase, the Control Agent is designed to fail safe. Rather than attempting an autonomous retry which could exacerbate instability, the system triggers a rollback mechanism to restore the previous known-good state and escalates the issue to a human operator. This design choice prioritizes network stability over autonomous persistence.

C. LESSONS LEARNED AND DEPLOYMENT INSIGHTS

The development of this framework revealed that the quality of the operator's input is fundamental. Vague use-case definitions or poorly defined KPIs lead to suboptimal designs, confirming the "garbage in, garbage out" principle. Therefore, operator training on how to formulate precise intents is a prerequisite for successful deployment.

From an engineering perspective, we found that implementing this framework required complex, iterative prompt engineering. The curated system prompts evolved to span hundreds of lines, incorporating explicit instructions to handle edge cases.

Furthermore, the orchestrator required significant logic to handle malformed LLM outputs. Despite instructions, models occasionally returned invalid response blocks, requiring a robust parsing layer to sanitize inputs before they could be consumed by the system.

Finally, we emphasize the importance of comprehensive logging and monitoring across all components, including the orchestrator and agents. To effectively debug and optimize the system, it is essential to capture instantiated prompts, raw model responses, HTTP status codes, and detailed performance metrics.

Our evaluation confirms that by combining Large Language Models with a well-designed agentic framework, DTN problems can be solved efficiently. However, a critical insight from our analysis is that newer models do not inherently guarantee better performance in specialized agentic workflows. This was starkly evident with OpenAI's GPT-5.2, released in December 2025 as a flagship model optimized for complex reasoning and coding [42]. Despite its superior benchmark specifications (e.g., SWE-bench [43] or GDPval [44]), it significantly underperformed compared to its predecessor (GPT-5) and competing models (e.g., Gemini-3-Pro) in our specific domain. We hypothesize that this discrepancy highlights a potential misalignment between general-purpose benchmarks and strict domain-specific constraints. Operators should not blindly upgrade to the latest model version but must continuously benchmark new releases against their specific domain constraints.

VIII. CONCLUSION AND FUTURE WORK

This paper presented an adaptable, agentic-AI framework that automates the entire lifecycle of Digital Twin Networks, from high-level intent formulation to deployment and operation. The framework's flexible design enables it to support various network use cases within the domain of core network management and topology optimization without requiring model fine-tuning or framework refactoring. We implemented an orchestrator which leveraged specialized agents, including those for use-case refinement, requirement engineering, perception, reflection, planning, code generation, validation, and control within a closed-loop architecture. This framework addresses the limitations of static, inflexible, time-consuming, and error-prone manually engineered DTNs. We introduced an Iterative AI-Automated Test-Driven Development methodology and reflection mechanisms to mitigate the risks of LLM hallucinations.

The extensive evaluation across two distinct 6G use cases, MEC service migration and network slicing, demonstrated the framework's ability to autonomously design, implement, test, validate, and deploy solutions on the physical network, leading to complete KPI fulfillment. The comparative analysis highlighted that while different LLMs may adopt varying engineering strategies, the framework's rigorous validation mechanisms ensure consistent adherence to operational objectives.

Future work will focus on addressing larger-scale, multi-domain, and more complex networks to further validate the framework's scalability and robustness in diverse 6G scenarios. Specifically, we aim to extend the experimental evaluation to other domains such as wireless (e.g., RAN optimization) and mobility management by integrating the corresponding domain models and prompts into the agent workspaces. In this regard, we started validating use cases regarding channel modeling using this framework and the preliminary results look promising. Ultimately, this work lays the foundation for self-evolving complex networks that

can dynamically adapt to the requirements of the future telecommunications landscape.

REFERENCES

- [1] M. Grieves, "Completing the cycle: Using PLM information in the sales and service functions," SME Management Forum, Troy, MI, USA, 2002.
- [2] *Digital Twin Network: Requirements and Architecture—Recommendation*, document ITU-T Y.3090, 2022. [Online]. Available: <https://handle.itu.int/11.1002/1000/14852>
- [3] P. Almasan et al., "Digital twin network: Opportunities and challenges," 2022, *arXiv:2201.01144*.
- [4] R. Khaldi, A. Lehmann, B. Ghita, and U. Trick, "Future BANs-DTN applications and requirements," in *Mobilkommunikation; 28. ITG-Fachtagung*, May 2024, pp. 70–75. [Online]. Available: <https://ieeexplore.ieee.org/document/10651584>
- [5] C. Zhou et al. (2025). *Network Digital Twin: Concepts and Reference Architecture*. Accessed: Dec. 13, 2025. [Online]. Available: <https://datatracker.ietf.org/doc/draft-irtf-nmrg-network-digital-twin-arch/10/>
- [6] E.-D. Jeong, H.-G. Kim, S. Nam, J.-H. Yoo, and J. W.-K. Hong, "S-switch: Switch configuration assistant with LLM and prompt engineering," in *Proc. IEEE Netw. Oper. Manage. Symp.*, May 2024, pp. 1–7, doi: [10.1109/NOMSS59830.2024.10575007](https://doi.org/10.1109/NOMSS59830.2024.10575007).
- [7] K. Dzevaroska and A. Leon-Garcia, "KPI assurance and LLMs for intent-based management," in *Proc. IEEE Netw. Oper. Manage. Symp.*, May 2025, pp. 1–9, doi: [10.1109/NOMSS57970.2025.11073595](https://doi.org/10.1109/NOMSS57970.2025.11073595).
- [8] M. Xu et al., "When large language model agents meet 6G networks: Perception, grounding, and alignment," *IEEE Wireless Commun.*, vol. 31, no. 6, pp. 63–71, Dec. 2024, doi: [10.1109/MWC.005.2400019](https://doi.org/10.1109/MWC.005.2400019).
- [9] Z. Chen, Q. Sun, N. Li, X. Li, Y. Wang, and I. Chih-Lin, "Enabling mobile AI agent in 6G era: Architecture and key technologies," *IEEE Netw.*, vol. 38, no. 5, pp. 66–75, Sep. 2024, doi: [10.1109/MNET.2024.3422309](https://doi.org/10.1109/MNET.2024.3422309).
- [10] Z. Xiao et al., "LLM agents as 6G orchestrator: A paradigm for task-oriented physical-layer automation," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2024, pp. 1–6, doi: [10.1109/GCWKSHP64532.2024.11101226](https://doi.org/10.1109/GCWKSHP64532.2024.11101226).
- [11] X. Xu et al., "Cross-domain orchestration with multi-agent LLM framework for enhanced task automation," in *Proc. Opt. Fiber Commun. Conf. (OFC)*, Mar. 2025, pp. 1–3. [Online]. Available: <https://ieeexplore.ieee.org/document/11047162>
- [12] H. Zaid, P. Safari, B. Shariati, A. Jafari, M. Balanici, and J. K. Fischer, "Multi-agent design for LLM-assisted network management," in *Proc. Opt. Fiber Commun. Conf. (OFC)*, Mar. 2025, pp. 1–3. [Online]. Available: <https://ieeexplore.ieee.org/document/11046468>
- [13] H. Cui, Y. Du, Q. Yang, Y. Shao, and S. C. Liew, "LLMind: Orchestrating AI and IoT with LLM for complex task execution," *IEEE Commun. Mag.*, vol. 63, no. 4, pp. 214–220, Apr. 2025, doi: [10.1109/MCOM.002.2400106](https://doi.org/10.1109/MCOM.002.2400106).
- [14] Z. Tao, W. Xu, Y. Huang, X. Wang, and X. You, "Wireless network digital twin for 6G: Generative AI as a key enabler," *IEEE Wireless Commun.*, vol. 31, no. 4, pp. 24–31, Aug. 2024, doi: [10.1109/MWC.002.2300564](https://doi.org/10.1109/MWC.002.2300564).
- [15] M. Shu, W. Sun, J. Zhang, X. Duan, and M. Ai, "Digital-twin-enabled 6G network autonomy and generative intelligence: Architecture, technologies and applications," *Digit. Twin*, vol. 2, p. 16, May 2025, doi: [10.12688/digitaltwin.17720.3](https://doi.org/10.12688/digitaltwin.17720.3).
- [16] M. Corici, H. Buhr, and T. Magedanz, "Generative twin for 6G and beyond 5G networks: Vision, challenges and architecture," in *Proc. 2nd Int. Conf. 6G Netw. (6GNet)*, Oct. 2023, pp. 1–6, doi: [10.1109/6GNET58894.2023.10317780](https://doi.org/10.1109/6GNET58894.2023.10317780).
- [17] Y. Hong, J. Wu, and R. Morello, "LLM-twin: Mini-giant model-driven beyond 5G digital twin networking framework with semantic secure communication and computation," *Sci. Rep.*, vol. 14, no. 1, p. 19065, Aug. 2024, doi: [10.1038/s41598-024-69474-5](https://doi.org/10.1038/s41598-024-69474-5).
- [18] S. Jiang, B. Lin, Y. Wu, and Y. Gao, "LINKs: Large language model integrated management for 6G empowered digital twin networks," in *Proc. IEEE 100th Veh. Technol. Conf. (VTC-Fall)*, Washington, DC, USA, Oct. 2024, pp. 1–6, doi: [10.1109/VTC2024-FALL63153.2024.10757470](https://doi.org/10.1109/VTC2024-FALL63153.2024.10757470).
- [19] Y. Yang, W. Sun, J. He, Y. Fu, and L. Xu, "Large generative model-enabled digital twin for 6G networks," *IEEE Netw.*, vol. 39, no. 3, pp. 29–36, May 2025, doi: [10.1109/MNET.2024.3520124](https://doi.org/10.1109/MNET.2024.3520124).
- [20] H. M. A. Zeeshan et al., "LLM-based retrieval-augmented generation: A novel framework for resource optimization in 6G and beyond wireless networks," *IEEE Commun. Mag.*, vol. 63, no. 10, pp. 60–67, Oct. 2025, doi: [10.1109/MCOM.001.2400774](https://doi.org/10.1109/MCOM.001.2400774).

- [21] M. Haider, I. Ahmed, Z. Hassan, K. Hasan, and H. Vincent Poor, "LLM-integrated digital twins for hierarchical resource allocation in 6G networks," 2025, *arXiv:2506.18293*.
- [22] S. Long et al., "6G comprehensive intelligence: Network operations and optimization based on large language models," *IEEE Netw.*, vol. 39, no. 4, pp. 192–201, Jul. 2025, doi: [10.1109/MNET.2024.3470774](https://doi.org/10.1109/MNET.2024.3470774).
- [23] N. Blasek, K. Eichenmüller, B. Ernst, N. Götz, B. Nast, and K. Sandkuhl, "Large language models in requirements engineering for digital twins," in *Proc. PoEM Companion*, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:268885083>
- [24] Y. Song et al., "Synergistic interplay of large language model and digital twin for autonomous optical networks: Field demonstrations," *IEEE Commun. Mag.*, vol. 63, no. 6, pp. 90–96, Jun. 2025, doi: [10.1109/MCOM.001.2400342](https://doi.org/10.1109/MCOM.001.2400342).
- [25] Y. Zhang et al., "GPT-enabled digital twin assistant for multi-task cooperative management in autonomous optical network," in *Proc. Opt. Fiber Commun. Conf. (OFC)*, Mar. 2024, pp. 1–3. [Online]. Available: <https://ieeexplore.ieee.org/document/10526863>
- [26] A. Zhou, Y. Song, Y. Zhang, M. Zhang, and D. Wang, "Large language model-driven AI agent in SDN controller towards intent-based management of optical networks," in *Proc. 50th Eur. Conf. Opt. Commun.*, Sep. 2024, pp. 1595–1598. [Online]. Available: <https://ieeexplore.ieee.org/document/10926080>
- [27] L. Yu et al., "ChannelGPT: A large model to generate digital twin channel for 6G environment intelligence," 2024, *arXiv:2410.13379*.
- [28] Y. Liu et al., "Generative AI in data center networking: Fundamentals, perspectives, and case study," *IEEE Netw.*, vol. 40, no. 2, pp. 165–174, Mar. 2026, doi: [10.1109/MNET.2025.3563262](https://doi.org/10.1109/MNET.2025.3563262).
- [29] M. Sheraz, T. C. Chuah, Y. L. Lee, M. M. Alam, A. Al-Habashna, and Z. Han, "A comprehensive survey on revolutionizing connectivity through artificial intelligence-enabled digital twin network in 6G," *IEEE Access*, vol. 12, pp. 49184–49215, 2024, doi: [10.1109/ACCESS.2024.3384272](https://doi.org/10.1109/ACCESS.2024.3384272).
- [30] T. Li et al., "Generative AI empowered network digital twins: Architecture, technologies, and applications," *ACM Comput. Surv.*, vol. 57, no. 6, pp. 1–43, Feb. 2025, doi: [10.1145/3711682](https://doi.org/10.1145/3711682).
- [31] B. Qin et al., "Machine and deep learning for digital twin networks: A survey," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34694–34716, Nov. 2024, doi: [10.1109/JIOT.2024.3416733](https://doi.org/10.1109/JIOT.2024.3416733).
- [32] H. Zhou et al., "Large language model (LLM) for telecommunications: A comprehensive survey on principles, key techniques, and opportunities," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 3, pp. 1955–2005, Jun. 2025, doi: [10.1109/COMST.2024.3465447](https://doi.org/10.1109/COMST.2024.3465447).
- [33] R. Khaldi, A. Lehmann, B. Ghita, and U. Trick, "Digital twin network models: A foundational knowledge base for agentic-LLM frameworks," 2026.
- [34] R. Khaldi, A. Lehmann, B. Ghita, and U. Trick, "DT-BAN design from a modeling perspective: A use case-driven approach," in *Proc. Mobilkom-munikat*; 29. ITG-Fachtagung, May 2025, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/11048367>
- [35] OpenAPI Initiative., *OpenAPI Specification*. Accessed: Dec. 13, 2025. [Online]. Available: <https://spec.openapis.org/oas/v3.2.0.html>
- [36] M. Peuster, H. Karl, and S. van Rossem, "MeDICINE: Rapid prototyping of production-ready network services in multi-PoP environments," in *Proc. IEEE Conf. Netw. Function Virtualization Softw. Defined Netw. (NFV-SDN)*, Nov. 2016, pp. 148–153, doi: [10.1109/NFV-SDN.2016.7919490](https://doi.org/10.1109/NFV-SDN.2016.7919490).
- [37] S. Ramírez. *FastAPI*. Accessed: Dec. 13, 2025. [Online]. Available: <https://fastapi.tiangolo.com/>
- [38] LangChain.LangGraph. Accessed: Dec. 13, 2025. [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview>
- [39] Meta.React. Accessed: Dec. 13, 2025. [Online]. Available: <https://react.dev/>
- [40] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. of California, Irvine, CA, USA, 2000. [Online]. Available: https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf
- [41] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, Mar. 2023, doi: [10.1145/3571730](https://doi.org/10.1145/3571730).
- [42] OpenAI., *Introducing GPT-5.2*. Accessed: Dec. 14, 2025. [Online]. Available: <https://openai.com/index/introducing-gpt-5-2/>
- [43] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. 12th Int. Conf. Learn. Represent.*, Dec. 2023. Accessed: Dec. 14, 2025. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQM66>
- [44] T. Patwardhan et al., "GDPval: Evaluating AI model performance on real-world economically valuable tasks," 2025, *arXiv:2510.04374*.
- [45] C. Rotsos et al., "Network service orchestration standardization: A technology survey," *Comput. Standards Interface*, vol. 54, pp. 203–215, Nov. 2017.
- [46] A. Kattepur, S. David, S. K. Mohalik, and S. Terrill, "Cognitive reasoning for 5G network lifecycle management," Accessed: Mar. 28, 2026. [Online]. Available: <https://www.ericsson.com/en/reports-and-papers/white-papers/cognitive-reasoning-for-5g-network-lifecycle-management>
- [47] C. Di Martino, U. Giordano, N. Mohanasamy, S. Russo, and M. Thottan, "In production performance testing of SDN control plane for telecom operators," in *Proc. 48th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Jun. 2018, pp. 642–653, doi: [10.1109/DSN.2018.00071](https://doi.org/10.1109/DSN.2018.00071).



RAMI KHALDI received the double bachelor's degree in computer science and business and economics from the Friedrich-Alexander University of Erlangen–Nuremberg, Germany, in 2018, and the master's degree in practical computer science from the University of Hagen, Germany, in 2022. He is currently a Software Integration Architect with Siemens and a Research Fellow with the Frankfurt University of Applied Sciences. Throughout his career, he has gained practical experience in software engineering, computer networking, and security. His primary research interests include digital twin networks, body area networks, and 6G.



ARMIN LEHMANN received the Dipl.-Ing. degree in information and communications technology from Frankfurt University of Applied Sciences, Germany, in 2005, and the Ph.D. degree from the University of Plymouth, U.K., in 2014. Since 2020, he has been Professor of programming in information technology with the Frankfurt University of Applied Sciences. His research interests include 5G/6G, digital twin networks, mission critical communications, NFV, M2M, and the IoT.



BOGDAN GHITA (Senior Member, IEEE) received the Ph.D. degree from the University of Plymouth, U.K., in 2005. He is currently a Professor with the University of Plymouth and leads the networking area with the Centre for Security, Communications, and Network Research. His research interests include computer networking and security, with a focus on network security, performance modeling and optimization, and wireless/mobile networking. He has published over 150 articles, has graduated 20 Ph.D. students, and has been a principal investigator in several industry-led, national, and EU research projects in these areas. He was a TPC member for over 100 international conference events and a reviewer of IEEE COMMUNICATIONS LETTERS, *Computer Communications*, and *Future Generation Computer Systems*.



ULRICH TRICK received the Dipl.-Ing. degree in electrical engineering-telecommunications and the Ph.D. degree from the University of Kaiserslautern, Germany, in 1983 and 1987, respectively. Since 2001, he has been a Professor of telecommunication networks with the Department for Computer Science and Engineering, Frankfurt University of Applied Sciences, Germany. His research interests include future networks, M2M, the IoT, WMN, virtualization, digital twin networks, 5G, and 6G.